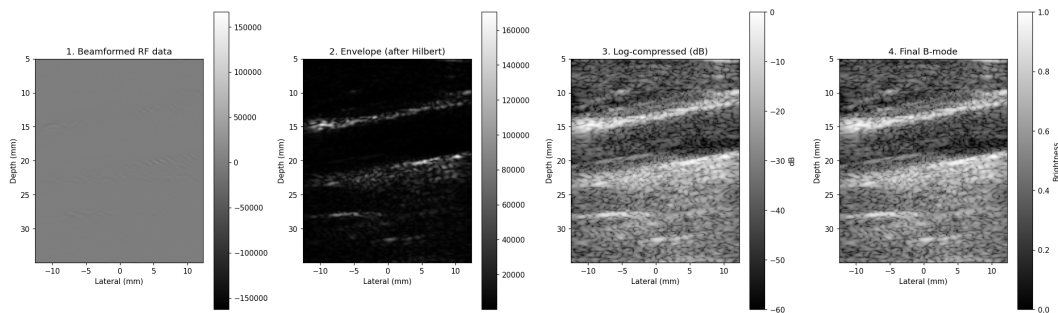


# Team 7: B-mode Image Formation

From Team 6's log-compressed echoes to the final grayscale ultrasound image

NITK-UsoundSim · 2-D ultrasound simulator · explanation, input, output and run guide

- Stage order: Image Reconstruction (Team 6: Hilbert envelope + log compression) → **B-mode Image Formation (this stage: dynamic range, grayscale brightness, axes, final image)** → Post-Image Processing (Team 8: despeckling).
- Input: Team 6's output files reconstructed\_carotid\_1/2.npz (log-compressed envelope in dB, 0 dB = peak, 1218 depth samples x 128 scan lines, real in-vivo carotid artery), and two simulated phantoms from the NITK-UsoundSim simulator (cyst lesion, point targets; 2078 x 256).
- Steps: clip to the dynamic range (60 dB), map to brightness 0..1, depth 5.0-35.0 mm and width -12.5-12.3 mm axes, display, save .npz/.png/\_axes.npz for Team 8.
- One command: **python3 run\_team7.py** (about 3 seconds). All numbers in this document come from that run.
- Check: this run's image matches the image the notebook saved earlier to 6.49e-06 (max difference).



carotid\_1, made by the team notebook in this run: RF and envelope (Team 6's arrays), then this stage's result: dB clipped to 60 dB and the final grayscale B-mode image.

## Contents

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>1 Overview</b>                                                 | <b>3</b>  |
| The team's work . . . . .                                         | 3         |
| Files in the folder . . . . .                                     | 3         |
| <b>2 Theory</b>                                                   | <b>4</b>  |
| 2.1 What arrives from Team 6 . . . . .                            | 4         |
| 2.2 Dynamic range and clipping . . . . .                          | 4         |
| 2.3 Mapping dB to brightness and grey levels . . . . .            | 4         |
| 2.4 Depth axis from the sample time . . . . .                     | 5         |
| 2.5 Lateral axis from the pitch . . . . .                         | 5         |
| 2.6 Display and aspect . . . . .                                  | 5         |
| 2.7 Saving for the next stage . . . . .                           | 6         |
| 2.8 What the carotid image shows . . . . .                        | 6         |
| <b>3 Code explanation</b>                                         | <b>7</b>  |
| 3.1 B_mode.ipynb: split between Team 6 and Team 7 . . . . .       | 7         |
| 3.2 Notebook code cells 10-12: the simulated phantoms . . . . .   | 8         |
| 3.3 How run_team7.py uses the team's own code . . . . .           | 9         |
| 3.4 run_team7.py (demo runner, written for this folder) . . . . . | 10        |
| 3.5 Full listings . . . . .                                       | 11        |
| <b>4 Input</b>                                                    | <b>20</b> |
| 4.1 The carotid files (Team 6's handoff) . . . . .                | 20        |
| 4.2 The simulated phantoms (cyst lesion, point targets) . . . . . | 20        |
| <b>5 Output</b>                                                   | <b>22</b> |
| 5.1 carotid_1 (60 dB) . . . . .                                   | 22        |
| 5.2 carotid_2 (60 dB) . . . . .                                   | 23        |
| 5.3 Effect of the dynamic range . . . . .                         | 24        |
| 5.4 Saved files for Team 8 . . . . .                              | 25        |
| 5.5 Reproducibility check . . . . .                               | 26        |
| 5.6 Simulated phantoms: cyst lesion and point targets . . . . .   | 27        |
| 5.7 Console output . . . . .                                      | 28        |
| <b>6 How to run</b>                                               | <b>34</b> |
| Troubleshooting . . . . .                                         | 34        |
| <b>7 Check yourself</b>                                           | <b>35</b> |

# 1 Overview

What this stage does and what is in the folder

An ultrasound scanner records echoes. Team 6 (Image Reconstruction) turns the beamformed RF echoes into **echo strength in decibels**: it detects the envelope with the Hilbert transform, normalises it to the strongest echo and applies log compression. Team 7 takes these dB values and forms the picture a doctor looks at: the **B-mode image** (B = brightness). Strong echoes become bright pixels, weak echoes dark pixels.

## Place in the pipeline

1. **Image Reconstruction (Team 6)**: Hilbert envelope, normalisation, log compression (dB).
2. **B-mode Image Formation (Team 7, this document)**: dynamic range (clipping), grayscale brightness 0..1, depth and width axes, display and saving of the final grayscale image.
3. **Post-Image Processing (Team 8)**: despeckling of the finished grayscale image.

## The team's work

- **B\_mode.ipynb**: the team notebook for **real in-vivo carotid artery** data (a blood vessel in the neck). Data source (as stated in the notebook): EPFL LTS5 public repository (us-non-stationary-deconv), 128-element linear probe, 5 MHz, one plane wave, delay-and-sum. The notebook holds the whole chain; from '5. APPLY DYNAMIC RANGE' on (plus cells 6-9) is this stage's work, run here. Code cells 1-9 are the team's original cells, unchanged; code cells 10-12 were added for the simulated phantoms.
- **input/reference/bmode\_carotid\_1.npy**: the image the notebook saved earlier. The runner compares this run's result with it to check that the result is reproducible.

## Files in the folder

| Path                                                    | What it is                                                                                   | Origin                                                           |
|---------------------------------------------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| README.md, requirements.txt                             | Run instructions; packages needed                                                            | written for this demo                                            |
| run_team7.py                                            | Runner; saves all results to output/                                                         | written for this demo                                            |
| code/B_mode.ipynb                                       | Team notebook, real carotid data (cells 10-12: simulated phantoms)                           | code cells 1-9: team original, unchanged; code cells 10-12 added |
| code/carotid_1_rf.npy                                   | Raw carotid RF, only for running B_mode.ipynb on its own                                     | copy of Team 6's raw input                                       |
| input/reconstructed_carotid_1.npz, _2.npz               | Log-compressed envelope and settings                                                         | output of Team 6, copied here as input                           |
| input/reconstructed_cyst_lesion.npz, _point_targets.npz | Simulated phantoms (same keys + depth_mm, width_mm)                                          | simulator Teams 1-5 + Team 6's steps                             |
| input/simulated/make_simulated_inputs.py                | Made the two simulated files (needs the main project, about 13 min; not needed for the demo) | written for this demo                                            |
| input/reference/bmode_carotid_1.npy, _axes.npz          | Image the notebook saved earlier                                                             | team original (for the check)                                    |
| output/; Team_7_..._Explanation.pdf                     | All results of run_team7.py; this document                                                   | made by the run; written for this demo                           |

## 2 Theory

From dB values to a grayscale picture, in easy words

Envelope detection with the Hilbert transform and log compression are Team 6's work; this stage starts from their result.

### 2.1 What arrives from Team 6

For every pixel (depth sample  $z$ , scan line  $x$ ) Team 6 delivers the echo strength in decibels relative to the strongest echo of the frame:

$$L(z, x) = 20 \log_{10} \left( \frac{E(z, x)}{\max E} + 10^{-12} \right) \text{ [dB]}, \quad L \leq 0$$

$E$  is Team 6's envelope. 0 dB = the strongest echo; -20 dB = 10 times weaker; -40 dB = 100 times weaker; -60 dB = 1000 times weaker.

In this run  $L$  goes from **-90.655 to 0 dB** for `carotid_1` and from **-86.1025 to 0 dB** for `carotid_2` (input file key `envelope_db`, not yet clipped). A screen cannot show 90 dB of detail usefully, and the very weakest values are mostly noise, so this stage chooses which part of the dB range to display.

### 2.2 Dynamic range and clipping

The **dynamic range** DR is how many dB below the peak are still shown. Every value weaker than -DR is set to -DR (it will be black); nothing is above 0 dB:

$$L_c(z, x) = \text{clip}(L(z, x), -DR, 0)$$

In code (notebook cell 5, section 5): `envelope_db = np.clip(envelope_db, -dynamic_range, 0)`

| Dynamic range            | carotid_1 black pixels | carotid_2 black pixels | Look of the image                                    |
|--------------------------|------------------------|------------------------|------------------------------------------------------|
| 40 dB                    | 26.44 %                | 30.11 %                | high contrast; tissue speckle and blood cut to black |
| 50 dB (project default)  | 4.90 %                 | 5.74 %                 | medium                                               |
| 60 dB (notebook setting) | 0.50 %                 | 0.63 %                 | softer and grayer; most weak detail visible          |

The percentages were measured in this run (Chapter 5.3 shows the images). Most pixels of the carotid lie between about -50 and -20 dB (histogram in Chapter 4), so moving the clip level from -60 to -40 dB turns a quarter of the image black.

### 2.3 Mapping dB to brightness and grey levels

$$B(z, x) = \frac{L_c(z, x) + DR}{DR} \in [0, 1], \quad G(z, x) = \text{round}(255 B(z, x)) \in \{0, \dots, 255\}$$

-DR dB becomes 0 (black), 0 dB becomes 1 (white); a straight line in between. In code (cell 5, section 6): `bmode = (envelope_db + dynamic_range) / dynamic_range`. The PNG saved by cell 9 stores 0..255 grey levels.

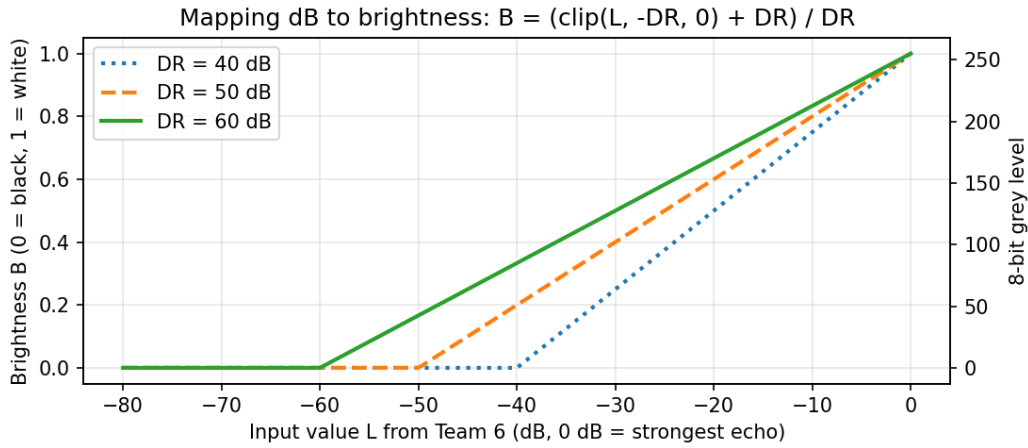


Illustration (made for this document): the gray mapping for three dynamic ranges. A smaller DR gives a steeper line: more contrast, but everything below  $-DR$  is black. Right axis: 8-bit grey level.

### Example

DR = 60 dB. A pixel at  $L = -30$  dB gets  $B = (-30 + 60)/60 = 0.5$ , grey level 128 (middle grey). A pixel at  $-75$  dB is clipped to  $-60$  dB and gets  $B = 0$  (black). The peak at  $0$  dB gets  $B = 1$  (white, 255).

In this run the mean brightness at 60 dB is 0.447 (mean grey level 114.1) for carotid\_1 and 0.415 (105.9) for carotid\_2.

## 2.4 Depth axis from the sample time

Row  $i$  of the image was recorded at time  $t = i / f_s$ . The sound goes down to the reflector and back up, so the reflector depth is half the distance travelled. The data starts at 5 mm:

$$z_i = z_0 + \frac{c t_i}{2} = z_0 + \frac{c i}{2 f_s}, \quad f_s = 31.25 \text{ MHz}, \quad c = 1540 \text{ m/s}, \quad z_0 = 5 \text{ mm}$$

One sample is  $1540 / (2 \times 31.25 \times 10^6) \text{ m} = 0.0246 \text{ mm}$  deep, so 1218 samples cover **5.0 to 35.0 mm**.

## 2.5 Lateral axis from the pitch

$$x_k = \left(k - \frac{N}{2}\right) \cdot \text{pitch}, \quad N = 128, \quad \text{pitch} = 0.1953 \text{ mm}$$

This gives **-12.5 to 12.3 mm** (the centre is half a pitch, about 0.1 mm, off zero; not visible).

## 2.6 Display and aspect

The image is shown with `imshow(..., cmap='gray', vmin=0, vmax=1, extent=[x_first, x_last, z_last, z_first])`. The extent puts real millimetres on both axes and puts depth 5 mm at the top (the probe is at the top). Because the extent is in mm and the default aspect is 'equal', 1 mm deep looks as long as 1 mm wide: the 30 mm x 24.8 mm field of view keeps its true shape, although the array has 1218 rows and only 128 columns. Fixing `vmin=0` and `vmax=1` means brightness 0 is always black and 1 always white.

## 2.7 Saving for the next stage

- **bmode\_<name>.npy**: brightness 0..1 as float32 (the exact values, for processing by Team 8).
- **bmode\_<name>.png**: the grayscale picture (plt.imsave, gray colour map, vmin 0, vmax 1).
- **bmode\_<name>\_axes.npz**: depth\_mm and width\_mm, so the next stage knows the pixel positions.

## 2.8 What the carotid image shows

The artery is seen along its length. Its **walls** are strong reflectors and appear as two bright, slanted bands (carotid\_1: the upper wall from about 15 mm depth on the left to about 10 mm on the right, the lower wall from about 21 mm to 19 mm). The **blood** reflects little and forms the darker band between them. The grainy texture is **speckle**, which Team 8 reduces afterwards.

## 3 Code explanation

Which code this stage runs, section by section

### 3.1 B\_mode.ipynb: split between Team 6 and Team 7

The team notebook contains the whole chain from RF to image. The runner splits it at the line # 5. *APPLY DYNAMIC RANGE* (code cell 5, line 16; the split is at the '=' comment line 15 just before it) and runs only this stage's part. The runner never edits the notebook file. Code cells 1-9 are the team's original cells, unchanged; code cells 10-12 were added for the simulated phantoms (section 3.2).

| Notebook code       | What it does                                                  | Who / run here?                                                                               |
|---------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Cell 1              | imports; DATA_FILE; np.load of the RF; prints                 | Team 6. Only the two lines 'import numpy as np' and 'import matplotlib.pyplot as plt' are run |
| Cell 2              | fs, f0, c, pitch, depth_start_mm, dynamic_range = 60          | Run (gives dynamic_range; re-assigns the same scalars as in the input file)                   |
| Cell 3              | Hilbert envelope                                              | Team 6, not run: envelope comes from the input file                                           |
| Cell 4              | Plot of one scan line: RF and envelope                        | Team 6, not run                                                                               |
| Cell 5, lines 1-14  | Sections 3-4: normalise, 20·log10                             | Team 6, not run: envelope_db comes from the input file                                        |
| Cell 5, lines 15-29 | Sections 5-6: clip to [-60, 0]; brightness (dB + 60)/60       | <b>Team 7, run</b>                                                                            |
| Cell 6              | Section 7: depth_mm, width_mm, extent                         | <b>Team 7, run</b>                                                                            |
| Cell 7              | Section 8: four panels RF, envelope, clipped dB, B-mode       | <b>Team 7, run</b> (RF and envelope panels show Team 6's arrays from the input)               |
| Cell 8              | Final B-mode figure                                           | <b>Team 7, run</b>                                                                            |
| Cell 9              | Section 9: save .numpy, .png, _axes.npz                       | <b>Team 7, run</b> inside output/                                                             |
| Cells 10-12 (added) | Simulated phantoms: dynamic range + brightness, display, save | <b>Team 7, run</b> on the two simulated files (section 3.2)                                   |

Before these pieces run, the runner loads the input file into the namespace (rf, envelope, envelope\_norm, envelope\_db, fs, f0, c, pitch, depth\_start\_mm) and sets *DATA\_FILE* = "carotid\_N\_rf.npy", only because cell 9 derives the output name (bmode\_carotid\_N) from it.

#### Section by section (the part run here)

- **Cell 5, section 5:** `np.clip(envelope_db, -dynamic_range, 0)` overwrites envelope\_db with the clipped values -60..0 dB (equation of 2.2).
- **Cell 5, section 6:** `bmode = (envelope_db + dynamic_range) / dynamic_range`; prints the shape (1218, 128) and the range 0.0 to 1.0 (equation of 2.3).
- **Cell 6:** sample index → time → depth (divide by 2) → mm, plus 5 mm start; width from the pitch; extent = [width first, width last, depth last, depth first] so depth increases downwards.
- **Cell 7:** one figure with four panels and colour bars. The dB panel uses vmin = -dynamic\_range, vmax = 0.
- **Cell 8:** the final image, 7 x 8 inch, gray, vmin 0, vmax 1, title '... (60 dB)'.
- **Cell 9:** name = DATA\_FILE without '\_rf.npy'; saves the three files for Team 8.

## 3.2 Notebook code cells 10-12: the simulated phantoms

At the end of the notebook, under the heading *Simulated phantoms from the NITK-UsoundSim simulator: cyst lesion and point targets*, one text cell and three code cells were added. They apply exactly this stage's steps to the two simulated input files (Chapter 4.2):

- **Cell 10:** SIM\_FILES points to ../input/reconstructed\_cyst\_lesion.npz and ../input/reconstructed\_point\_targets.npz. For each file:  $np.clip(envelope\_db, -dynamic\_range, 0)$  (section 5) and  $(dB + dynamic\_range) / dynamic\_range$  (section 6), with  $dynamic\_range = 60$  from cell 2. The depth and width axes are taken from the file (depth\_mm, width\_mm), because the simulator's settings differ from the carotid data. Prints shape (2078, 256), brightness 0.0 to 1.0, depth 0.0 to 40.0 mm, width -10.0 to 10.0 mm.
- **Cell 11:** both images side by side, gray, vmin 0, vmax 1, extent in mm, aspect 'equal' (saved by the runner as simulated\_01\_cyst\_lesion\_and\_point\_targets.png).
- **Cell 12:** saves bmode\_cyst\_lesion.npy / .png / \_axes.npz and bmode\_point\_targets.npy / .png / \_axes.npz, the same three kinds of file as cell 9.

The paths start with ../input/ because the notebook sits in code/. The runner executes cells 10 and 12 with the working directory output/, so ../input/ again points to the folder's input/ and the files are saved in output/.

### B\_mode.ipynb - code cell 10 (added for the simulated phantoms; run here)

```

1 # =====
2 # 10. SIMULATED PHANTOMS: DYNAMIC RANGE AND BRIGHTNESS
3 # =====
4
5 SIM_FILES = {
6     "cyst_lesion":  "../input/reconstructed_cyst_lesion.npz",
7     "point_targets": "../input/reconstructed_point_targets.npz",
8 }
9
10 sim = {}
11 for sim_name, sim_path in SIM_FILES.items():
12     d = np.load(sim_path)
13
14     sim_db = np.clip(d["envelope_db"], -dynamic_range, 0)           # 5. apply dynamic range
15     sim_bmode = (sim_db + dynamic_range) / dynamic_range           # 6. dB to brightness (0 to 1)
16
17     sim_depth_mm = d["depth_mm"]                                   # real depth axis (mm)
18     sim_width_mm = d["width_mm"]                                   # real scan-line positions (mm)
19     sim_extent = [sim_width_mm[0], sim_width_mm[-1], sim_depth_mm[-1], sim_depth_mm[0]]
20
21     sim[sim_name] = {"bmode": sim_bmode, "depth_mm": sim_depth_mm,
22                     "width_mm": sim_width_mm, "extent": sim_extent}
23
24     print(sim_name)
25     print("  B-mode shape:", sim_bmode.shape, "(depth samples x scan lines)")
26     print("  Brightness range:", sim_bmode.min(), "to", sim_bmode.max())
27     print("  Depth: %.1f mm to %.1f mm" % (sim_depth_mm[0], sim_depth_mm[-1]))
28     print("  Width: %.1f mm to %.1f mm" % (sim_width_mm[0], sim_width_mm[-1]))

```

### B\_mode.ipynb - code cell 11 (added for the simulated phantoms; run here)

```

1 # =====
2 # 11. DISPLAY THE SIMULATED B-MODE IMAGES
3 # =====
4
5 fig, ax = plt.subplots(1, 2, figsize=(12, 8))
6
7 titles = {"cyst_lesion": "Cyst lesion (6 mm anechoic cyst at 30 mm)",
8          "point_targets": "Point targets (10, 20, 30 mm depth)"}
9
10 for a, sim_name in zip(ax, ["cyst_lesion", "point_targets"]):
11     im = a.imshow(sim[sim_name]["bmode"], cmap="gray", vmin=0, vmax=1,
12                  extent=sim[sim_name]["extent"], aspect="equal")
13     a.set_title(f"Simulated B-mode: {titles[sim_name]}\n({dynamic_range} dB)")
14     a.set_xlabel("Lateral (mm)")

```

```

15     a.set_ylabel("Depth (mm)")
16     plt.colorbar(im, ax=a, label="Brightness")
17
18 plt.tight_layout()
19 plt.show()

```

### B\_mode.ipynb - code cell 12 (added for the simulated phantoms; run here)

```

1 # =====
2 # 12. SAVE THE SIMULATED B-MODE IMAGES FOR POST-PROCESSING
3 # =====
4
5 for sim_name in sim:
6     np.save(f"bmode_{sim_name}.npz", sim[sim_name]["bmode"].astype(np.float32))
7     plt.imshow(f"bmode_{sim_name}.png", sim[sim_name]["bmode"], cmap="gray", vmin=0, vmax=1)
8     np.savez(f"bmode_{sim_name}_axes.npz", depth_mm=sim[sim_name]["depth_mm"],
9             width_mm=sim[sim_name]["width_mm"])
10    print("Saved:", f"bmode_{sim_name}.npz", "|", f"bmode_{sim_name}.png", "|", f"bmode_{sim_name}"
        _axes.npz")

```

## 3.3 How run\_team7.py uses the team's own code

**The algorithm is the team's own work.** Dynamic range clipping, the dB-to-brightness grayscale mapping and forming and saving the B-mode image all come from the team notebook

**code/B\_mode.ipynb.** run\_team7.py reads the notebook's code cells and executes this stage's cells one by one, **unchanged** (the notebook file is never edited), starting from Team 6's handoff files input/reconstructed\_carotid\_1.npz and \_2.npz, and then the added code cells 10-12 on the two simulated files. Code cells 1-9 are the team's original cells, unchanged; code cells 10-12 were added for the simulated phantoms. run\_team7.py itself is only a demo wrapper around this code.

### Where run\_team7.py uses your code

| run_team7.py line(s) | Notebook code cell used                                                                                | What it does there                                                                                          |
|----------------------|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| 120-123, 141-142     | all code cells (read only)                                                                             | code_cells() reads B_mode.ipynb as JSON and takes the source text of every code cell                        |
| 148-149, 232         | Cell 1, only lines 1-2 (import numpy as np, import matplotlib.pyplot as plt)                           | Runs the two import lines; the rest of cell 1 (np.load of the raw RF) is not run                            |
| 233                  | Cell 2 (settings)                                                                                      | Sets fs, f0, c, pitch, depth_start_mm and dynamic_range = 60                                                |
| 143-146, 234         | Cell 5, second half: from the '=' line before '# 5. APPLY DYNAMIC RANGE' to the end (cell lines 15-29) | Clips envelope_db to -60..0 dB and maps it to brightness bmode = (dB + 60)/60                               |
| 235                  | Cell 6                                                                                                 | Depth and width axes in mm, extent                                                                          |
| 236                  | Cell 7                                                                                                 | Figure with all steps (saved as carotid_N_01_all_steps.png)                                                 |
| 237                  | Cell 8                                                                                                 | Final B-mode figure (saved as carotid_N_02_final_bmode.png)                                                 |
| 238                  | Cell 9                                                                                                 | Saves bmode_carotid_N.npz / .png / _axes.npz (run inside output/)                                           |
| 252-253              | Cell 1 (the two import lines) and cell 2, again                                                        | Part 2.3: imports and dynamic_range = 60 in a fresh namespace for the simulated phantoms                    |
| 254                  | Cell 10 (added)                                                                                        | Loads the two simulated files, clips to -60..0 dB, brightness 0..1, axes from the file (run inside output/) |

| run_team7.py line(s) | Notebook code cell used | What it does there                                                                           |
|----------------------|-------------------------|----------------------------------------------------------------------------------------------|
| 255                  | Cell 11 (added)         | Both simulated images side by side (saved as simulated_01_cyst_lesion_and_point_targets.png) |
| 256                  | Cell 12 (added)         | Saves bmode_cyst_lesion.* and bmode_point_targets.* (run inside output/)                     |

Lines 232-238 are inside the loop that starts at line 223, so the same cells run once for carotid\_1 and once for carotid\_2. Lines 252-256 (Part 2.3, lines 245-257) run once, after that loop. Each piece is executed by run\_source() (lines 126-135) in a shared namespace (one per frame, one for the simulated phantoms).

### What run\_team7.py adds on top (written for this demo, not the team's code)

| run_team7.py line(s) | What it adds                                                                                                                                                                                                                        |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 69-84, 370-372       | Tee: prints to the screen and keeps a copy of the console text, saved as output/console_log.txt                                                                                                                                     |
| 105-117              | Replaces plt.show() by a function that saves each notebook figure as a PNG in output/                                                                                                                                               |
| 151-162, 231         | load_handoff(): loads Team 6's .npz into the notebook's variable names (rf, envelope, envelope_norm, envelope_db, fs, f0, c, pitch, depth_start_mm) and sets DATA_FILE = "carotid_N_rf.npy" (only used by cell 9 to name the files) |
| 178-216              | Part 1: prints every key of both input files; figure input_envelope_db_overview.png (dB images + histogram)                                                                                                                         |
| 245-251              | Part 2.3 set-up: banner, figure name prefix 'simulated' and a fresh namespace for notebook cells 10-12                                                                                                                              |
| 262-291              | Part 3: 40 / 50 / 60 dB comparison written again with the notebook's formula (line 270); figure dynamic_range_40_50_60dB.png; black-pixel percentages; line 289 checks that its 60 dB image equals the notebook's bmode             |
| 296-333              | Part 4: reproducibility check of output/bmode_carotid_1.npy against input/reference/bmode_carotid_1.npy (max and mean difference, 8-bit grey levels, axes); figure reproducibility_check_60dB.png                                   |
| 338-365              | Part 5: summary, timings and list of output files; lines 350-352 print one line per simulated image                                                                                                                                 |
| 374-394              | Console pictures console_screenshot_1.png, _2.png, ... (75 lines each)                                                                                                                                                              |

**To see it yourself:** open run\_team7.py at the lines above; open code/B\_mode.ipynb to see the same cells.

## 3.4 run\_team7.py (demo runner, written for this folder)

- Uses matplotlib's file-only mode (Agg) and replaces *plt.show()* by a function that saves each figure in output/.
- Part 1: prints every key of both input files and draws the runner figure input\_envelope\_db\_overview.png.
- Part 2: reads B\_mode.ipynb as JSON, splits cell 5 at the '=' line before '# 5. APPLY DYNAMIC RANGE' and executes the pieces listed in 3.1 in one namespace (cell 9 with working directory output/), for carotid\_1 and carotid\_2.
- Part 2.3: runs the cell 1 import lines, cell 2 and the added code cells 10, 11, 12 on the simulated cyst lesion and point targets (cells 10 and 12 with working directory output/).
- Part 3: runner figure dynamic\_range\_40\_50\_60dB.png using the notebook's formula on the input envelope\_db; prints mean brightness and black-pixel percentages; confirms that its 60 dB image

is identical to the notebook's bmode.

- Part 4: reproducibility check: loads this run's output/bmode\_carotid\_1.npy and the earlier saved input/reference/bmode\_carotid\_1.npy, prints the largest and mean absolute difference, compares the 8-bit grey levels and the axes, and draws reproducibility\_check\_60dB.png.
- Part 5: summary and timings; saves console\_log.txt and console pictures (its own small function, nothing imported from outside the folder).

### 3.5 Full listings

**B\_mode.ipynb - code cell 1 (Team 6's cell; only the two import lines (1-2) are run here)**

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.signal import hilbert
4
5
6 # =====
7 # 1. LOAD REAL RF DATA
8 # =====
9
10 DATA_FILE = "carotid_1_rf.npy"      # change to "carotid_2_rf.npy" for the second image
11
12 rf = np.load(DATA_FILE)              # shape: (depth samples, scan lines)
13
14 print("RF data shape:", rf.shape)
15 print("Depth samples:", rf.shape[0])
16 print("Scan lines   :", rf.shape[1])
17 print("Minimum:", rf.min())
18 print("Maximum:", rf.max())

```

**B\_mode.ipynb - code cell 2 (run here)**

```

1 # =====
2 # SAMPLING AND PROBE INFORMATION (real probe)
3 # =====
4
5 fs = 31.25e6          # Sampling frequency = 31.25 MHz
6 f0 = 5e6              # Centre frequency of probe = 5 MHz
7 c = 1540              # Speed of sound in tissue = 1540 m/s
8 pitch = 0.1953e-3     # Distance between scan lines = 0.1953 mm
9 depth_start_mm = 5.0  # First sample is at 5 mm depth
10
11 dynamic_range = 60    # 60 dB

```

**B\_mode.ipynb - code cell 3 (Team 6, not run here)**

```

1 # =====
2 # 2. ENVELOPE DETECTION (HILBERT TRANSFORM)
3 # =====
4 # Hilbert transform is applied along depth (axis=0) for every scan line.
5 # abs() of the analytic signal gives the envelope (echo strength).
6
7 analytic_signal = hilbert(rf, axis=0)
8 envelope = np.abs(analytic_signal)
9
10 print("Envelope shape:", envelope.shape)
11 print("Minimum:", envelope.min())
12 print("Maximum:", envelope.max())

```

**B\_mode.ipynb - code cell 4 (Team 6, not run here)**

```

1 # =====
2 # SHOW ONE SCAN LINE: RF SIGNAL AND ITS ENVELOPE
3 # =====
4
5 line = rf.shape[1] // 2 # middle scan line
6 depth_axis_mm = depth_start_mm + (np.arange(rf.shape[0]) / fs) * c / 2 * 1000
7
8 plt.figure(figsize=(12, 4))
9 plt.plot(depth_axis_mm, rf[:, line], linewidth=0.7, label="RF signal")
10 plt.plot(depth_axis_mm, envelope[:, line], linewidth=1.5, label="Envelope (Hilbert)")
11 plt.xlabel("Depth (mm)")
12 plt.ylabel("Amplitude")
13 plt.title(f"Scan line {line}: RF signal and envelope")
14 plt.legend()
15 plt.show()

```

**B\_mode.ipynb code cell 5:** lines 1-14 are Team 6's (not run here); **lines 15-29 are run here.** Shown as two listings; the second keeps its own numbering from 1 (= cell lines 15-29).

**B\_mode.ipynb - code cell 5, lines 1-14 (Team 6, not run here)**

```

1 # =====
2 # 3. NORMALIZE ENVELOPE
3 # =====
4
5 envelope_norm = envelope / np.max(envelope)
6
7
8 # =====
9 # 4. LOG COMPRESSION (CONVERT TO dB)
10 # =====
11
12 envelope_db = 20 * np.log10(envelope_norm + 1e-12)
13

```

**B\_mode.ipynb - code cell 5, lines 15-29 (run here)**

```

1 # =====
2 # 5. APPLY DYNAMIC RANGE
3 # =====
4
5 envelope_db = np.clip(envelope_db, -dynamic_range, 0)
6
7
8 # =====
9 # 6. CONVERT dB TO BRIGHTNESS (0 to 1)
10 # =====
11
12 bmode = (envelope_db + dynamic_range) / dynamic_range
13
14 print("B-mode shape:", bmode.shape)
15 print("Brightness range:", bmode.min(), "to", bmode.max())

```

**B\_mode.ipynb - code cell 6 (run here)**

```

1 # =====
2 # 7. CALCULATE DEPTH AND WIDTH (mm)
3 # =====
4
5 num_samples, num_scanlines = rf.shape
6
7 sample_indices = np.arange(num_samples)
8 time = sample_indices / fs # time of each sample (s)
9 depth = (c * time) / 2 # divide by 2: sound goes and comes back
10 depth_mm = depth_start_mm + depth * 1000
11
12 width_mm = (np.arange(num_scanlines) - num_scanlines / 2) * pitch * 1000
13
14 print("Depth: %.1f mm to %.1f mm" % (depth_mm[0], depth_mm[-1]))
15 print("Width: %.1f mm to %.1f mm" % (width_mm[0], width_mm[-1]))

```

```

16
17 extent = [width_mm[0], width_mm[-1], depth_mm[-1], depth_mm[0]]

```

### B\_mode.ipynb - code cell 7 (run here)

```

1 # =====
2 # 8. DISPLAY ALL STEPS
3 # =====
4
5 fig, ax = plt.subplots(1, 4, figsize=(20, 6))
6
7 im = ax[0].imshow(rf, cmap="gray", extent=extent)
8 ax[0].set_title("1. Beamformed RF data")
9 plt.colorbar(im, ax=ax[0])
10
11 im = ax[1].imshow(envelope, cmap="gray", extent=extent)
12 ax[1].set_title("2. Envelope (after Hilbert)")
13 plt.colorbar(im, ax=ax[1])
14
15 im = ax[2].imshow(envelope_db, cmap="gray", vmin=-dynamic_range, vmax=0, extent=extent)
16 ax[2].set_title("3. Log-compressed (dB)")
17 plt.colorbar(im, ax=ax[2], label="dB")
18
19 im = ax[3].imshow(bmode, cmap="gray", vmin=0, vmax=1, extent=extent)
20 ax[3].set_title("4. Final B-mode")
21 plt.colorbar(im, ax=ax[3], label="Brightness")
22
23 for a in ax:
24     a.set_xlabel("Lateral (mm)")
25     a.set_ylabel("Depth (mm)")
26
27 plt.tight_layout()
28 plt.show()

```

### B\_mode.ipynb - code cell 8 (run here)

```

1 # =====
2 # FINAL B-MODE IMAGE
3 # =====
4
5 plt.figure(figsize=(7, 8))
6
7 plt.imshow(bmode, cmap="gray", vmin=0, vmax=1, extent=extent)
8
9 plt.xlabel("Lateral (mm)")
10 plt.ylabel("Depth (mm)")
11 plt.title("Final B-mode Ultrasound Image\nReal in-vivo carotid artery (60 dB)")
12 plt.colorbar(label="Brightness")
13
14 plt.show()

```

### B\_mode.ipynb - code cell 9 (run here (inside output/))

```

1 # =====
2 # 9. SAVE OUTPUT FOR IMAGE RECONSTRUCTION / POST-PROCESSING TEAM
3 # =====
4
5 name = DATA_FILE.replace("_rf.npy", "")
6
7 np.save(f"bmode_{name}.npy", bmode.astype(np.float32)) # float values 0 to 1
8 plt.imsave(f"bmode_{name}.png", bmode, cmap="gray", vmin=0, vmax=1) # grayscale image
9 np.savez(f"bmode_{name}_axes.npz", depth_mm=depth_mm, width_mm=width_mm)
10
11 print("Saved:", f"bmode_{name}.npy", "|", f"bmode_{name}.png", "|", f"bmode_{name}_axes.npz")
12 print("Image size:", bmode.shape, "(depth samples x scan lines)")

```

Code cells 10-12 (added for the simulated phantoms) are listed in section 3.2.

**run\_team7.py (runner)**

```

1  """
2  run_team7.py - Team 7: B-mode Image Formation (demo runner)
3
4  One command runs this stage and saves every result in output/:
5
6      python3 run_team7.py      (macOS / Linux)
7      python  run_team7.py      (Windows)
8
9  Place in the simulator
10   Team 6 (Image Reconstruction): Hilbert envelope, normalisation, log compression (dB)
11   -> Team 7 (THIS STAGE): dynamic range (clipping), grayscale brightness 0..1,
12       depth/width axes, display and saving of the final grayscale B-mode image
13   -> Team 8 (Post-Image Processing): despeckling of the finished image
14
15  Input
16   input/reconstructed_carotid_1.npz, input/reconstructed_carotid_2.npz
17   = the output of Team 6 (Image Reconstruction), copied here as input.
18   input/reconstructed_cyst_lesion.npz, input/reconstructed_point_targets.npz
19   = simulated phantoms from our simulator (Teams 1-5) + Team 6's steps, same keys
20   plus depth_mm and width_mm (made by input/simulated/make_simulated_inputs.py).
21   Keys: rf, envelope, envelope_norm, envelope_db (dB, 0 dB = peak, not yet clipped),
22         fs, f0, c, pitch, depth_start_mm.
23
24  What it does
25   Part 1 Shows the input (Team 6's log-compressed envelope in dB).
26   Part 2 Runs the B-mode part of code/B_mode.ipynb on carotid_1 and carotid_2:
27         code cell 2 (settings, gives dynamic_range), code cell 5 from section
28         "5. APPLY DYNAMIC RANGE" to its end (sections 5 and 6), code cells 6, 7, 8, 9.
29         Code cells 1, 3, 4 and the first half of cell 5 (sections 1 to 4: load RF,
30         Hilbert envelope, normalise, log compression) are Team 6's work; their
31         results come from the input file instead.
32         Then code cells 10, 11, 12 (added to the notebook for the simulated
33         phantoms): the same dynamic range and brightness steps on the cyst lesion
34         and the point targets, display and saving.
35   Part 3 Dynamic-range comparison (40 / 50 / 60 dB) made from the input envelope_db.
36   Part 4 Reproducibility check: this run's output/bmode_carotid_1.npy against the image
37         the notebook saved earlier (input/reference/bmode_carotid_1.npy).
38   Part 5 Summary, timings, console log and console pictures.
39
40  code/B_mode.ipynb is the team's notebook: code cells 1-9 are the team's original cells,
41  unchanged; code cells 10-12 were added for the simulated phantoms. This runner only reads it.
42  """
43
44  import io
45  import json
46  import os
47  import sys
48  import time
49  from pathlib import Path
50
51  sys.dont_write_bytecode = True # keep code/ clean (no __pycache__)
52
53  import matplotlib # noqa: E402
54
55  matplotlib.use("Agg") # save figures to files, no windows
56  import matplotlib.pyplot as plt # noqa: E402
57  import numpy as np # noqa: E402
58
59  ROOT = Path(__file__).resolve().parent
60  CODE = ROOT / "code"
61  INPUT = ROOT / "input"
62  OUTPUT = ROOT / "output"
63  OUTPUT.mkdir(exist_ok=True)
64
65
66  # -----
67  # Console: print to the screen AND keep a copy for output/console_log.txt
68  # -----
69  class Tee(io.TextIOBase):
70      def __init__(self, stream):
71          self.stream = stream
72          self.buffer_text = io.StringIO()
73
74      def write(self, s):
75          self.stream.write(s)
76          self.buffer_text.write(s)

```

```

77         return len(s)
78
79     def flush(self):
80         self.stream.flush()
81
82
83 tee = Tee(sys.stdout)
84 sys.stdout = tee
85
86
87 def banner(text):
88     print()
89     print("=" * 78)
90     print(text)
91     print("=" * 78)
92
93
94 def rng(a):
95     return f"{float(np.min(a)):.6g} to {float(np.max(a)):.6g}"
96
97
98 # -----
99 # plt.show() replacement: save each figure to output/ with a numbered name
100 # -----
101 saved_figures = []
102 _fig_state = {"prefix": "fig", "names": [], "count": 0}
103
104
105 def _save_and_close(*args, **kwargs):
106     _fig_state["count"] += 1
107     n = _fig_state["count"]
108     names = _fig_state["names"]
109     label = names[n - 1] if n <= len(names) else f"figure{n}"
110     path = OUTPUT / f"{_fig_state['prefix']}_{n:02d}_{label}.png"
111     plt.gcf().savefig(path, dpi=110, bbox_inches="tight")
112     plt.close("all")
113     saved_figures.append(path.name)
114     print(f"    [figure saved] output/{path.name}")
115
116
117 plt.show = _save_and_close
118
119
120 def code_cells(notebook_path):
121     """Return the source text of every code cell of a Jupyter notebook."""
122     nb = json.loads(Path(notebook_path).read_text(encoding="utf-8"))
123     return ["".join(c["source"]) for c in nb["cells"] if c["cell_type"] == "code"]
124
125
126 def run_source(src, label, namespace, workdir=None):
127     """Execute one piece of notebook code in the shared namespace."""
128     print(f"--- {label}")
129     start_dir = os.getcwd()
130     try:
131         if workdir is not None:
132             os.chdir(workdir)
133         exec(compile(src, label, "exec"), namespace)
134     finally:
135         os.chdir(start_dir)
136
137
138 # -----
139 # Split code/B_mode.ipynb into Team 6's part and this stage's part
140 # -----
141 NB = CODE / "B_mode.ipynb"
142 cells = code_cells(NB)
143 MARK = "# 5. APPLY DYNAMIC RANGE"
144 cell5 = cells[4]
145 cut = cell5.rindex("# ===", 0, cell5.index(MARK)) # the '=' line just before the marker
146 CELL5_TEAM7 = cell5[cut:] # sections 5 and 6 of code cell 5
147 # cell 1's import lines only (NOT its np.load): numpy and matplotlib.pyplot
148 IMPORTS = "\n".join(ln for ln in cells[0].splitlines()
149                     if ln.startswith("import numpy") or ln.startswith("import matplotlib.pyplot"))
150
151 HANDOFF_KEYS = ["rf", "envelope", "envelope_norm", "envelope_db", "fs", "f0", "c", "pitch", "depth",
152                  "← _start_mm"]

```

```

153
154 def load_handoff(n):
155     """Load Team 6's handoff file into a fresh namespace (arrays + Python scalars)."""
156     d = np.load(INPUT / f"reconstructed_carotid_{n}.npz")
157     ns = {"__name__": "__main__"}
158     for k in HANDOFF_KEYS:
159         v = d[k]
160         ns[k] = v.item() if v.ndim == 0 else v
161     ns["DATA_FILE"] = f"carotid_{n}_rf.npy" # cell 9 derives the output name from it
162     return ns
163
164
165 timings = {}
166 t_total = time.perf_counter()
167
168 banner("TEAM 7 - B-MODE IMAGE FORMATION (NITK-UsoundSim)")
169 print("Stage order: Team 6 Image Reconstruction (Hilbert envelope + log compression)")
170 print("    -> Team 7 B-mode Image Formation (THIS STAGE: dynamic range, gray levels, axes, f
    ↳ inal image)")
171 print("    -> Team 8 Post-Image Processing (despeckling)")
172 print("Folder :", ROOT.name)
173 print("Python :", sys.version.split()[0], "| numpy", np.__version__, "| matplotlib", matplotlib.__
    ↳ version__)
174
175 # -----
176 # PART 1: the input (Team 6's handoff)
177 # -----
178 banner("PART 1: INPUT = output of Team 6 (Image Reconstruction), copied into input/")
179 t = time.perf_counter()
180 handoff = {}
181 for n in (1, 2):
182     d = np.load(INPUT / f"reconstructed_carotid_{n}.npz")
183     handoff[n] = d
184     print(f"input/reconstructed_carotid_{n}.npz")
185     for k in HANDOFF_KEYS:
186         v = d[k]
187         if v.ndim == 0:
188             print(f" {k:<15s} scalar {v.dtype} = {v.item()}")
189         else:
190             print(f" {k:<15s} shape {v.shape}, {v.dtype}, values {rng(v)}")
191
192 fig, ax = plt.subplots(1, 3, figsize=(16, 6), gridspec_kw={"width_ratios": [1, 1, 1.25]})
193 for k, n in enumerate((1, 2)):
194     d = handoff[n]
195     L = d["envelope_db"]
196     nz, nx = L.shape
197     dmm = float(d["depth_start_mm"]) + np.arange(nz) / float(d["fs"]) * float(d["c"]) / 2 * 1000
198     wmm = (np.arange(nx) - nx / 2) * float(d["pitch"]) * 1000
199     im = ax[k].imshow(L, cmap="gray", vmin=float(L.min()), vmax=0, extent=[wmm[0], wmm[-1], dmm[-1]
    ↳ ], dmm[0]))
200     ax[k].set_title(f"reconstructed_carotid_{n}.npz: envelope_db\n(full range {L.min():.1f} to 0 d
    ↳ B, not clipped)")
201     ax[k].set_xlabel("Lateral (mm)")
202     ax[k].set_ylabel("Depth (mm)")
203     plt.colorbar(im, ax=ax[k], label="dB (0 dB = peak)")
204     ax[2].hist(L.ravel(), bins=180, histtype="step", lw=1.2, label=f"carotid_{n}")
205     for dr, ls in [(40, "-"), (50, "--"), (60, "-.")]:
206         ax[2].axvline(-dr, color="k", ls=ls, lw=1, label=f"--{dr} dB (clip level for DR = {dr} dB)")
207     ax[2].set_xlabel("envelope_db value (dB)")
208     ax[2].set_ylabel("Number of pixels")
209     ax[2].set_title("Histogram of the input dB values")
210     ax[2].legend(fontsize=8)
211 fig.tight_layout()
212 fig.savefig(OUTPUT / "input_envelope_db_overview.png", dpi=110)
213 plt.close(fig)
214 saved_figures.append("input_envelope_db_overview.png")
215 print("    [figure saved] output/input_envelope_db_overview.png (runner figure)")
216 timings["Part 1 input overview"] = time.perf_counter() - t
217
218 # -----
219 # PART 2: this stage's notebook code on the handoff data
220 # -----
221 carotid_figs = ["all_steps", "final_bmode"]
222 ns_all = {}
223 for n in (1, 2):
224     banner(f"PART 2.{n}: code/B_mode.ipynb, B-mode part, on reconstructed_carotid_{n}.npz")
225     print("Loaded into the namespace from the input file: " + ", ".join(HANDOFF_KEYS))

```

```

226     print(f'Runner sets DATA_FILE = "carotid_{n}_rf.npy" (only used by cell 9 to name the saved fi
↳ les).')
227     print("Not run here (Team 6's work): cell 1 np.load, cell 3 Hilbert, cell 4 scan-line plot,")
228     print("cell 5 sections 3-4 (normalise, log compression).")
229     t = time.perf_counter()
230     _fig_state.update(prefix=f"carotid_{n}", names=carotid_figs, count=0)
231     ns = load_handoff(n)
232     run_source(IMPORTS, "cell 1, import lines only: " + IMPORTS.replace("\n", "; "), ns)
233     run_source(cells[1], "B_mode.ipynb code cell 2 (settings; gives dynamic_range)", ns)
234     run_source(CELL5_TEAM7, "B_mode.ipynb code cell 5, sections 5-6 (clip to dynamic range, bright
↳ ness 0..1)", ns)
235     run_source(cells[5], "B_mode.ipynb code cell 6 (depth and width axes)", ns)
236     run_source(cells[6], "B_mode.ipynb code cell 7 (display all steps)", ns)
237     run_source(cells[7], "B_mode.ipynb code cell 8 (final B-mode image)", ns)
238     run_source(cells[8], "B_mode.ipynb code cell 9 (save, run inside output/)", ns, workdir=OUTPUT
↳ )
239     ns_all[n] = ns
240     timings[f"Part 2.{n} notebook stage, carotid_{n}"] = time.perf_counter() - t
241
242     # -----
243     # PART 2.3: the notebook's simulated-phantom cells (cyst lesion, point targets)
244     # -----
245     banner("PART 2.3: code/B_mode.ipynb code cells 10-12 on the simulated phantoms")
246     print("Input: input/reconstructed_cyst_lesion.npz, input/reconstructed_point_targets.npz")
247     print("(made by our simulator, Teams 1-5, plus Team 6's envelope and log compression;")
248     print(" see input/simulated/make_simulated_inputs.py)")
249     t = time.perf_counter()
250     _fig_state.update(prefix="simulated", names=["cyst_lesion_and_point_targets"], count=0)
251     ns_sim = {"__name__": "__main__"}
252     run_source(IMPORTS, "cell 1, import lines only: " + IMPORTS.replace("\n", "; "), ns_sim)
253     run_source(cells[1], "B_mode.ipynb code cell 2 (settings; gives dynamic_range)", ns_sim)
254     run_source(cells[9], "B_mode.ipynb code cell 10 (simulated: dynamic range, brightness)", ns_sim, w
↳ orkdir=OUTPUT)
255     run_source(cells[10], "B_mode.ipynb code cell 11 (display the simulated images)", ns_sim)
256     run_source(cells[11], "B_mode.ipynb code cell 12 (save, run inside output/)", ns_sim, workdir=OUTP
↳ UT)
257     timings["Part 2.3 notebook, simulated phantoms"] = time.perf_counter() - t
258
259     # -----
260     # PART 3: dynamic range comparison (runner figure, the notebook's formula)
261     # -----
262     banner("PART 3: effect of the dynamic range (runner figure, notebook formula)")
263     print("For DR = 40, 50, 60 dB: B = (clip(envelope_db, -DR, 0) + DR) / DR from the input envelope
↳ _db")
264     t = time.perf_counter()
265     dr_images = {}
266     black = {}
267     for n in (1, 2):
268         L = handoff[n]["envelope_db"]
269         for dr in (40, 50, 60):
270             b = (np.clip(L, -dr, 0) + dr) / dr
271             dr_images[(n, dr)] = b
272             black[(n, dr)] = 100.0 * np.mean(b == 0)
273             print(f" carotid_{n}, DR {dr} dB: mean brightness {b.mean():.3f}, "
274                   f"black (clipped) pixels {black[(n, dr)]:5.2f} %, pixels at full white {np.sum(b ==
↳ 1)}")
275     ext = ns_all[1]["extent"]
276     fig, ax = plt.subplots(2, 3, figsize=(12, 10.5))
277     for r, n in enumerate((1, 2)):
278         for k, dr in enumerate((40, 50, 60)):
279             im = ax[r, k].imshow(dr_images[(n, dr)], cmap="gray", vmin=0, vmax=1, extent=ext)
280             ax[r, k].set_title(f"carotid_{n}, DR {dr} dB\n(black pixels {black[(n, dr)]:.1f} %)")
281             ax[r, k].set_xlabel("Lateral (mm)")
282             ax[r, k].set_ylabel("Depth (mm)")
283     fig.colorbar(im, ax=ax, fraction=0.025, label="Brightness (0 = black, 1 = white)")
284     fig.suptitle("Runner figure: dynamic range 40 / 50 / 60 dB applied to Team 6's envelope_db")
285     fig.savefig(OUTPUT / "dynamic_range_40_50_60dB.png", dpi=110, bbox_inches="tight")
286     plt.close(fig)
287     saved_figures.append("dynamic_range_40_50_60dB.png")
288     print(" [figure saved] output/dynamic_range_40_50_60dB.png (runner figure)")
289     same60 = np.array_equal(dr_images[(1, 60)], ns_all[1]["bmode"])
290     print(f" 60 dB image here identical to the notebook's bmode for carotid_1: {same60}")
291     timings["Part 3 dynamic range comparison"] = time.perf_counter() - t
292
293     # -----
294     # PART 4: reproducibility check against the notebook's earlier saved image
295     # -----

```

```

296 banner("PART 4: reproducibility check (this run vs the image the notebook saved earlier)")
297 t = time.perf_counter()
298 ref = np.load(INPUT / "reference" / "bmode_carotid_1.npy")
299 ref_axes = np.load(INPUT / "reference" / "bmode_carotid_1_axes.npz")
300 saved_today = np.load(OUTPUT / "bmode_carotid_1.npy")
301 axes_today = np.load(OUTPUT / "bmode_carotid_1_axes.npz")
302 diff = np.abs(saved_today.astype(np.float64) - ref.astype(np.float64))
303 d_nb = float(diff.max())
304 axes_same = all(np.array_equal(ref_axes[k], axes_today[k]) for k in ("depth_mm", "width_mm"))
305 gray_diff = np.abs(np.round(255 * saved_today.astype(np.float64)) - np.round(255 * ref.astype(np.f
    ↳ loat64)))
306 print(f" this run output/bmode_carotid_1.npy : shape {saved_today.shape}, {saved_today.
    ↳ dtype}, values {rng(saved_today)}")
307 print(f" reference input/reference/bmode_carotid_1.npy : shape {ref.shape}, {ref.dtype}, values {
    ↳ rng(ref)}")
308 print(f" max |this run - reference| = {d_nb:.3g}")
309 print(f" mean |this run - reference| = {float(diff.mean()):.3g}")
310 print(f" 8-bit gray levels round(255 * B): {int(np.sum(gray_diff > 0))} of {gray_diff.size} pixel
    ↳ s differ, "
311 f"by at most {int(gray_diff.max())} level (values that sit right on a rounding boundary)")
312 print(f" depth_mm and width_mm identical to reference : {axes_same}")
313 ok = d_nb < 1e-5 and axes_same
314 print(" RESULT:", "PASS - same image (difference is float32 rounding, far below one gray level 1/
    ↳ 255 = 0.0039)"
315 if ok else "FAIL - results differ")
316 timings["Part 4 reproducibility check"] = time.perf_counter() - t
317
318 fig, ax = plt.subplots(1, 3, figsize=(13, 6))
319 for a, img, title in [(ax[0], ref, "Saved reference\n(input/reference/bmode_carotid_1.npy)"),
320 (ax[1], saved_today, "This run\n(output/bmode_carotid_1.npy, 60 dB)"]):
321     a.imshow(img, cmap="gray", vmin=0, vmax=1, extent=ext)
322     a.set_title(title)
323     a.set_xlabel("Lateral (mm)")
324     a.set_ylabel("Depth (mm)")
325 im = ax[2].imshow(diff, cmap="magma", extent=ext)
326 ax[2].set_title(f"|this run - reference|\n(max {d_nb:.2g})")
327 ax[2].set_xlabel("Lateral (mm)")
328 plt.colorbar(im, ax=ax[2])
329 fig.tight_layout()
330 fig.savefig(OUTPUT / "reproducibility_check_60dB.png", dpi=110)
331 plt.close(fig)
332 saved_figures.append("reproducibility_check_60dB.png")
333 print(" [figure saved] output/reproducibility_check_60dB.png (runner figure)")
334
335 # -----
336 # PART 6: summary
337 # -----
338 banner("PART 5: SUMMARY")
339 for n in (1, 2):
340     ns = ns_all[n]
341     L_in = handoff[n]["envelope_db"]
342     print(f"carotid_{n}:")
343     print(f" input envelope_db {L_in.shape}, {L_in.dtype}, values {rng(L_in)} dB")
344     print(f" dynamic range {ns['dynamic_range']} dB -> clipped dB values {rng(ns['envelope_
    ↳ db'])} dB")
345     print(f" B-mode brightness {ns['bmode'].dtype}, values {rng(ns['bmode'])}, mean {ns['bmode']
    ↳ .mean():.3f}")
346     print(f" 8-bit gray levels 0 to 255 (round(255 * B)): mean {np.round(255 * ns['bmode']).mean
    ↳ ():.1f}")
347     print(f" axes depth {ns['depth_mm'][0]:.1f} to {ns['depth_mm'][-1]:.1f} mm, "
348 f"lateral {ns['width_mm'][0]:.1f} to {ns['width_mm'][-1]:.1f} mm")
349     print(" black pixels " + ", ".join(f"{dr} dB: {black[(n, dr)]:.2f} %" for dr in (40, 50
    ↳ , 60)))
350 for sim_name, s in ns_sim["sim"].items():
351     print(f"{sim_name}: B-mode {s['bmode'].shape}, brightness {rng(s['bmode'])}, mean {s['bmode'].
    ↳ mean():.3f}, "
352 f"depth {s['depth_mm'][0]:.1f} to {s['depth_mm'][-1]:.1f} mm, lateral {s['width_mm'][0]:
    ↳ .1f} to {s['width_mm'][-1]:.1f} mm")
353 print(f"Reproducibility (carotid_1): max |this run - reference| = {d_nb:.3g} ({'PASS' if ok else
    ↳ 'FAIL'})")
354 print("\nTimings:")
355 for k, v in timings.items():
356     print(f" {k:<40s} {v:6.2f} s")
357 total = time.perf_counter() - t_total
358 print(f" {'TOTAL':<40s} {total:6.2f} s")
359
360 print("\nFiles written to output:/")

```

```

361 for f in sorted(p.name for p in OUTPUT.iterdir() if not p.name.startswith("console_")):
362     print(" ", f)
363 print(" console_log.txt")
364 print(" console_screenshot_1.png, _2.png, ... (pictures of this console text, 75 lines each)")
365 print("\nDone.")
366
367 # -----
368 # Save the console text and a screenshot-style picture of it
369 # -----
370 sys.stdout = tee.stream
371 text = tee.buffer_text.getvalue()
372 (OUTPUT / "console_log.txt").write_text(text, encoding="utf-8")
373
374 lines = []
375 for line in text.splitlines():
376     while len(line) > 125:
377         lines.append(line[:125])
378         line = " " + line[125:]
379     lines.append(line)
380 # one picture per 75 lines, so each picture fits on a page
381 chunks = [lines[i:i + 75] for i in range(0, len(lines), 75)]
382 shots = []
383 for k, chunk in enumerate(chunks, start=1):
384     h = 0.6 + 0.165 * len(chunk)
385     fig = plt.figure(figsize=(13.5, h), facecolor="#1e1e1e")
386     fig.text(0.01, 1 - 0.25 / h, f"Terminal - python3 run_team7.py (part {k} of {len(chunks)}",
387             color="#9cdcf",
388             family="monospace", fontsize=9, va="top", weight="bold")
389     fig.text(0.01, 1 - 0.55 / h, "\n".join(chunk), color="#e8e8e8", family="monospace",
390             fontsize=8.2, va="top", linespacing=1.25)
391     name = f"console_screenshot_{k}.png"
392     fig.savefig(OUTPUT / name, dpi=100, facecolor="#1e1e1e")
393     plt.close(fig)
394     shots.append(name)
395 print("Console log saved to output/console_log.txt and output/" + " ", output/"\n".join(shots))

```

## 4 Input

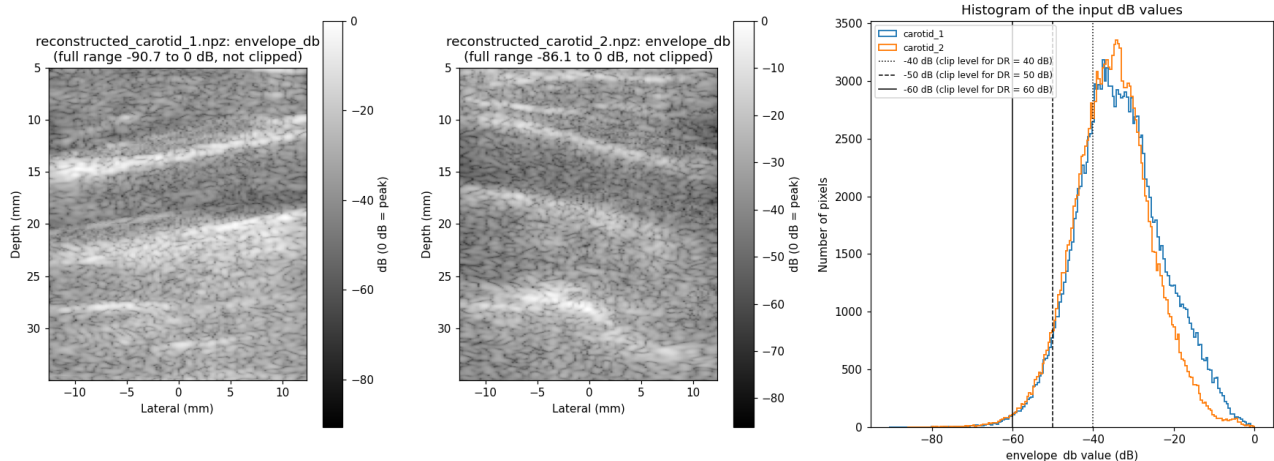
Team 6's handoff files and the two simulated phantoms

### 4.1 The carotid files (Team 6's handoff)

The input files **input/reconstructed\_carotid\_1.npz** and **input/reconstructed\_carotid\_2.npz** are the output of Team 6 (Image Reconstruction), copied here as input. They were written with `np.savez` and hold exactly the arrays of the team notebook after its sections 1-4 (load RF, Hilbert envelope, normalise, log compression). Values from this run:

| Key                | Shape, dtype        | carotid_1              | carotid_2               |
|--------------------|---------------------|------------------------|-------------------------|
| rf                 | (1218, 128) float32 | -162577 to 166418      | -130418 to 139393       |
| envelope           | (1218, 128) float32 | 4.99583 to 170356      | 7.11421 to 143632       |
| envelope_norm      | (1218, 128) float32 | 2.93259e-05 to 1       | 4.95308e-05 to 1        |
| <b>envelope_db</b> | (1218, 128) float32 | <b>-90.655 to 0 dB</b> | <b>-86.1025 to 0 dB</b> |
| fs                 | scalar float64      | 31250000.0 (31.25 MHz) | same                    |
| f0                 | scalar float64      | 5000000.0 (5 MHz)      | same                    |
| c                  | scalar float64      | 1540.0 (m/s)           | same                    |
| pitch              | scalar float64      | 0.0001953 (m)          | same                    |
| depth_start_mm     | scalar float64      | 5.0                    | same                    |

**envelope\_db** is the key this stage works on: dB relative to the frame peak (0 dB = peak), not yet clipped. **rf** and **envelope** are used only for the first two panels of the notebook's all-steps figure.



Runner figure `input_envelope_db_overview.png`: `envelope_db` of both frames on their full dB range, and a histogram of all pixel values with the clip levels -40, -50 and -60 dB. Pixels left of a line become black for that dynamic range.

**Reference file:** `input/reference/bmode_carotid_1.npy` (+ `_axes.npz`) is the image the team notebook saved earlier; it is used only for the reproducibility check.

### 4.2 The simulated phantoms (cyst lesion, point targets)

**input/reconstructed\_cyst\_lesion.npz** and **input/reconstructed\_point\_targets.npz** come from the full NITK-UsoundSim simulator: Teams 1-5 (phantom, propagation, tissue interaction, receive delay-and-sum beamforming), followed by Team 6's steps with the same formulas as the notebook (Hilbert envelope, normalise to the peak, 20 log10). They were made by

**input/simulated/make\_simulated\_inputs.py**, which needs the main NITK-UsoundSim project to run again (about 13 minutes); it is not needed for the demo, the .npz files are already in input/. The files have the same keys as Team 6's handoff plus the real axes `depth_mm` and `width_mm`.

- **cyst\_lesion**: speckle tissue with a 6 mm radius **anechoic cyst** (lesion) centred at 30 mm depth. In the simulator's phantom the scatterers inside the cyst have amplitude 0, so it should appear as a dark circle.
- **point\_targets**: 3 point reflectors at 10, 20 and 30 mm depth, at  $x = -5, 0$  and  $+5$  mm. They should appear as small bright dots.

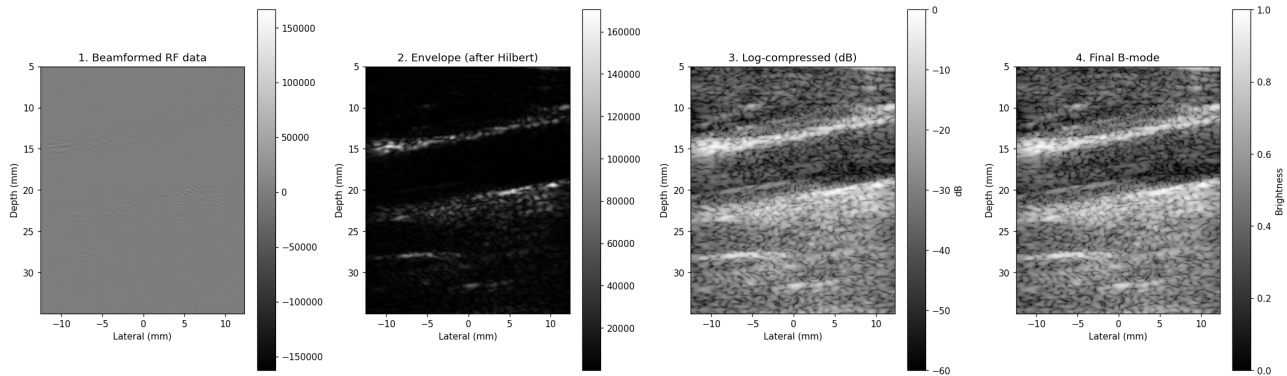
| Key                | Shape, dtype        | cyst_lesion                         | point_targets           |
|--------------------|---------------------|-------------------------------------|-------------------------|
| rf                 | (2078, 256) float32 | -13.4753 to 14.4873                 | -1.25105 to 1.11094     |
| envelope           | (2078, 256) float32 | 0.000262872 to 15.0736              | 5.47296e-11 to 1.27506  |
| envelope_norm      | (2078, 256) float32 | 1.74392e-05 to 1                    | 4.29232e-11 to 1        |
| <b>envelope_db</b> | (2078, 256) float32 | <b>-95.1694 to 0 dB</b>             | <b>-207.146 to 0 dB</b> |
| fs                 | scalar float64      | 40000000.0 (40 MHz)                 | same                    |
| f0                 | scalar float64      | 8000000.0 (8 MHz)                   | same                    |
| c                  | scalar float64      | 1540.0 (m/s)                        | same                    |
| pitch              | scalar float64      | 7.84314e-05 (m) = scan-line spacing | same                    |
| depth_start_mm     | scalar float64      | 0.0                                 | same                    |
| depth_mm           | (2078,) float64     | 0 to 39.9823 mm                     | same                    |
| width_mm           | (256,) float64      | -10 to 10 mm (256 scan lines)       | same                    |

The point-target file goes down to about -207 dB because almost the whole image is empty: far from the three reflectors the envelope is nearly zero. The 60 dB dynamic range sets all of that to black (88.0 % of the pixels of `output/bmode_point_targets.npy` are exactly 0).

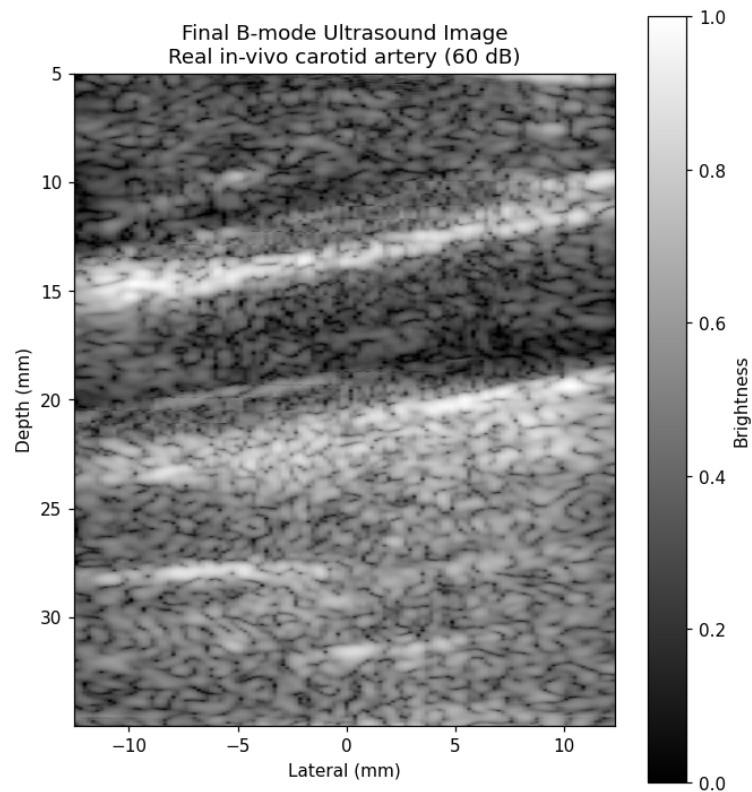
## 5 Output

Every figure, the saved images and the real console output

### 5.1 carotid\_1 (60 dB)

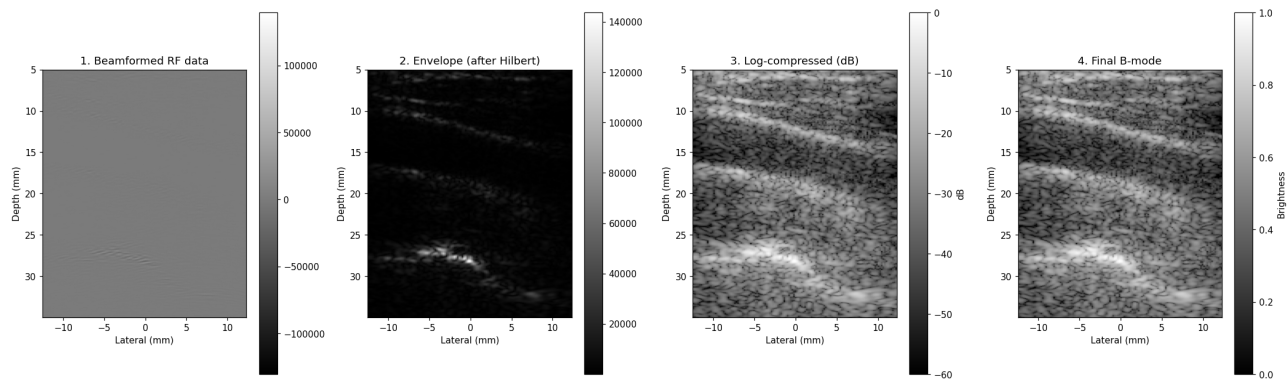


carotid\_1\_01\_all\_steps.png (notebook cell 7): (1) the RF and (2) the envelope, both from Team 6's file; (3) the dB values after clipping to -60..0 dB; (4) the brightness 0..1. Panels 3 and 4 look the same because the gray mapping is a straight line; they differ only in the colour-bar units.

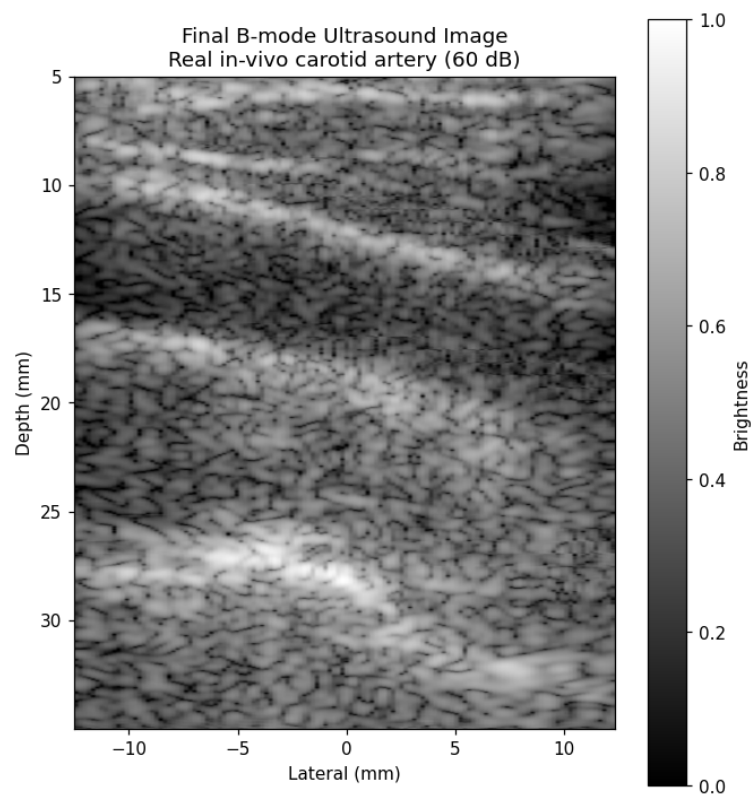


carotid\_1\_02\_final\_bmode.png (cell 8): the final B-mode image. Bright slanted bands: vessel walls; darker band between them: blood in the lumen; grainy texture: speckle.

## 5.2 carotid\_2 (60 dB)



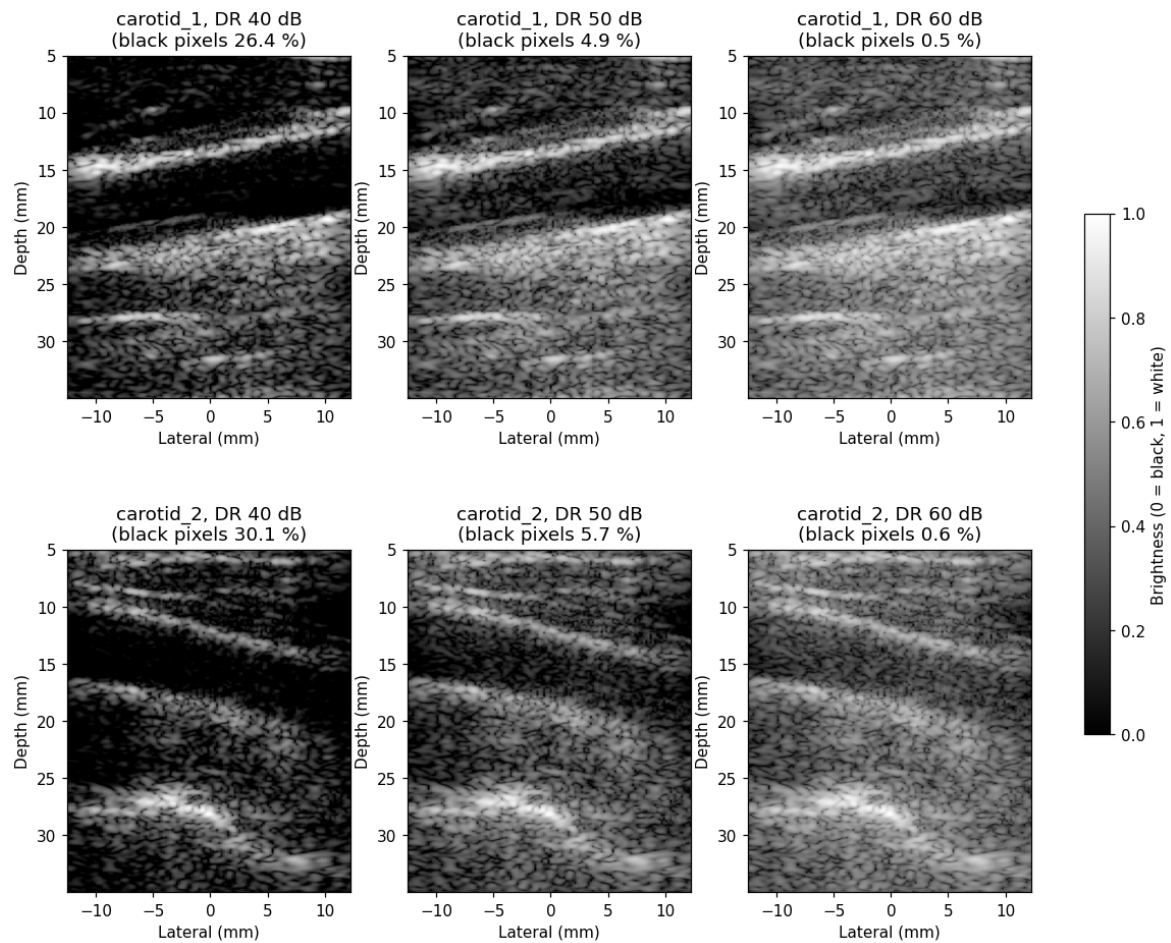
carotid\_2\_01\_all\_steps.png: the same four panels for the second frame.



carotid\_2\_02\_final\_bmode.png: final B-mode image of the second frame.

## 5.3 Effect of the dynamic range

Runner figure: dynamic range 40 / 50 / 60 dB applied to Team 6's envelope\_db

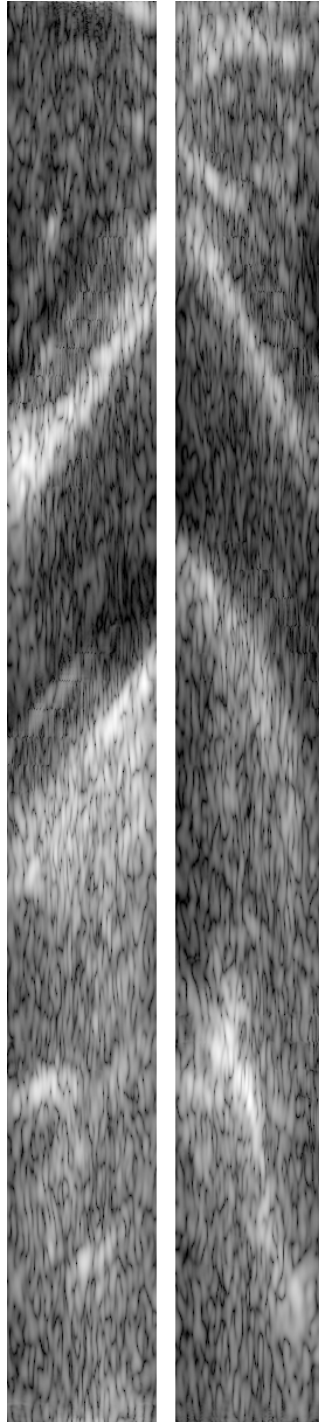


Runner figure dynamic\_range\_40\_50\_60dB.png:  $B = (\text{clip}(\text{envelope\_db}, -\text{DR}, 0) + \text{DR}) / \text{DR}$  for DR = 40, 50, 60 dB (the notebook's own formula). The runner confirmed that its 60 dB image is identical to the notebook's bmode.

| Frame     | DR    | Mean brightness | Black (clipped) pixels |
|-----------|-------|-----------------|------------------------|
| carotid_1 | 40 dB | 0.210           | 26.44 %                |
| carotid_1 | 50 dB | 0.341           | 4.90 %                 |
| carotid_1 | 60 dB | 0.447           | 0.50 %                 |
| carotid_2 | 40 dB | 0.168           | 30.11 %                |
| carotid_2 | 50 dB | 0.303           | 5.74 %                 |
| carotid_2 | 60 dB | 0.415           | 0.63 %                 |

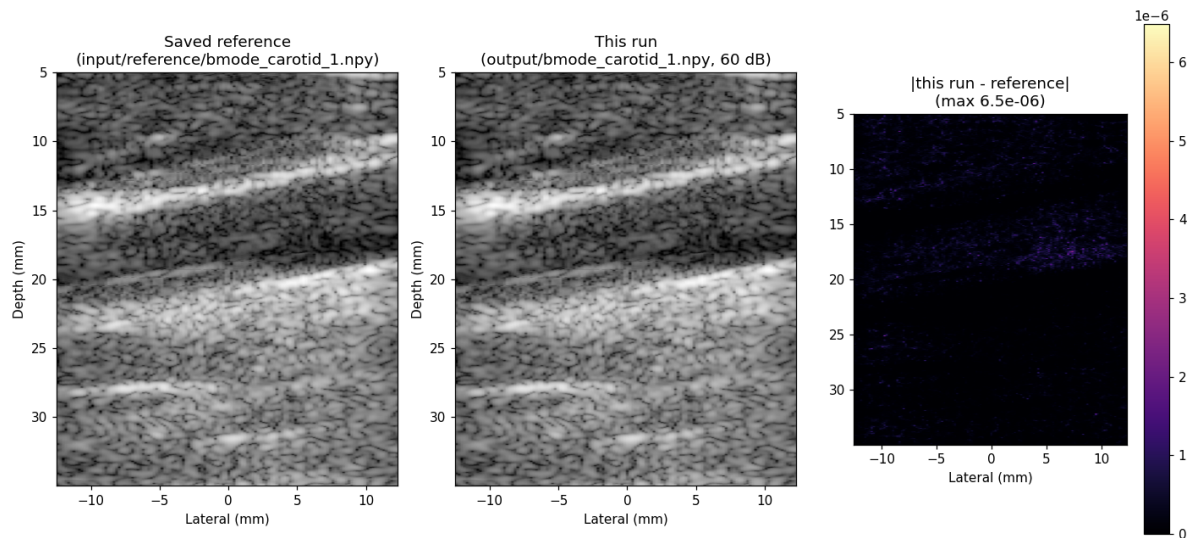
## 5.4 Saved files for Team 8

Cell 9 wrote **bmode\_carotid\_1.npy / .png / \_axes.npz** and **bmode\_carotid\_2.npy / .png / \_axes.npz** into output/: brightness (1218, 128) float32 in 0..1, the grayscale PNG and the axes (depth 5.0 to 35.0 mm, width -12.5 to 12.3 mm). The two saved PNGs (one pixel per sample, so they look tall and narrow):



bmode\_carotid\_1.png (left) and bmode\_carotid\_2.png (right) exactly as saved: 128 x 1218 pixels each, grey levels 0-255. Displayed with the mm axes (as in 5.1/5.2), the image gets its true shape.

## 5.5 Reproducibility check

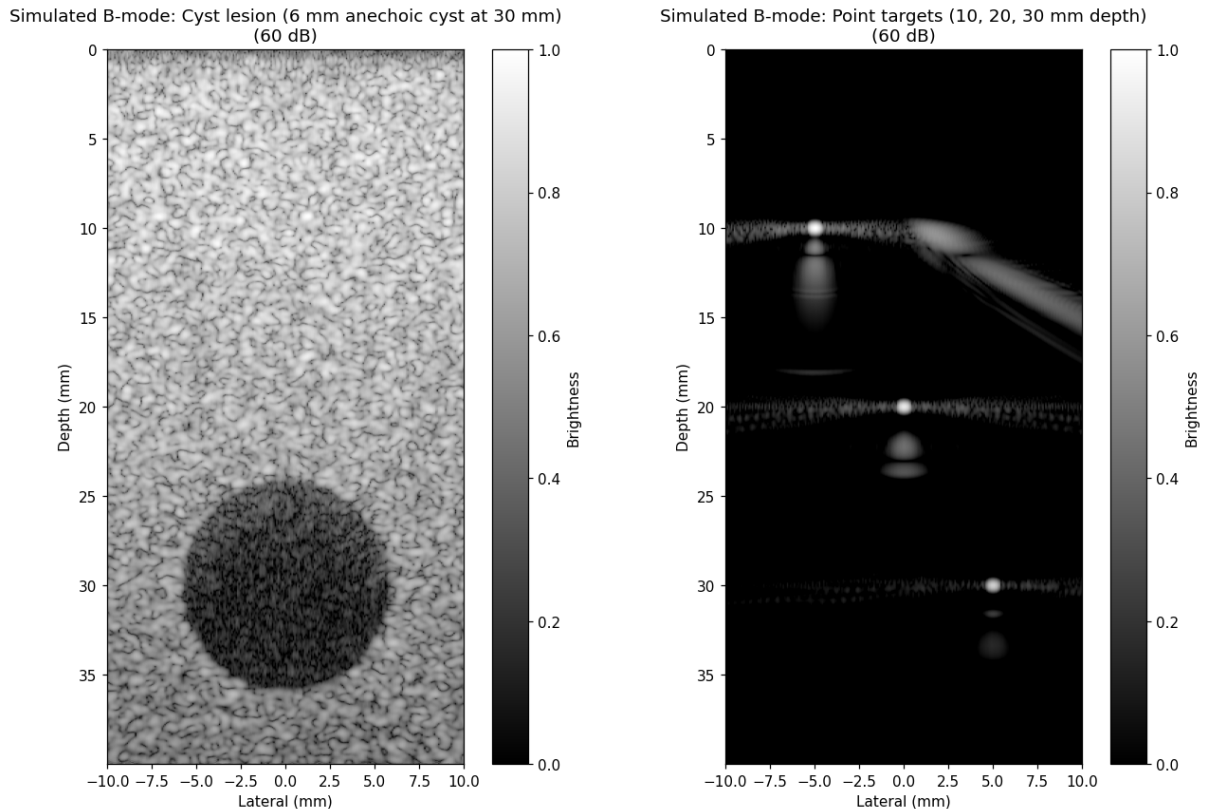


Runner figure reproducibility\_check\_60dB.png: the reference image saved earlier by the notebook (input/reference/bmode\_carotid\_1.npy), this run's output/bmode\_carotid\_1.npy, and |this run - reference|.

| Comparison (carotid_1, 60 dB): this run vs reference | Result                                |
|------------------------------------------------------|---------------------------------------|
| Max  difference  of the brightness 0..1              | 6.49e-06                              |
| Mean  difference  of the brightness 0..1             | 1.11e-07                              |
| 8-bit grey levels round(255 B)                       | 6 of 155904 pixels differ, by 1 level |
| depth_mm and width_mm axes                           | identical                             |

One grey level is  $1/255 = 0.0039$ , so the two images are the same picture. The tiny differences are float32 rounding differences between the team's earlier notebook run (which started from the raw RF) and this run (which starts from the envelope\_db in Team 6's file). The 6 pixels that change by one grey level sit exactly on a rounding boundary.

## 5.6 Simulated phantoms: cyst lesion and point targets



simulated\_01\_cyst\_lesion\_and\_point\_targets.png (notebook cell 11): the simulated cyst lesion (left) and point targets (right) after this stage's steps, 60 dB dynamic range, depth 0 to 40 mm, width -10 to +10 mm.

- **Cyst lesion:** the cyst is the dark circle at about 24-36 mm depth and  $x = -6$  to  $+6$  mm (radius 6 mm around 30 mm depth), inside the grainy speckle of the tissue. It is dark grey rather than pure black: measured on output/bmode\_cyst\_lesion.npy, the mean brightness within 5 mm of the cyst centre is 0.18, against 0.67 in the tissue more than 8 mm from the centre (below 5 mm depth). Mean brightness of the whole image: 0.599.
- **Point targets:** the three bright dots at  $(x, \text{depth}) = (-5, 10)$ ,  $(0, 20)$  and  $(+5, 30)$  mm are the reflectors. The rest of the image is almost black (mean brightness 0.022).
- The faint streaks and blobs near the points (short lines at the depth of each point, small blobs below them and the slanted band on the upper right) are artefacts of the simulator itself, not of this stage: they are also visible in the main NITK-UsoundSim project's results/point\_targets.png. They look stronger here because 60 dB dynamic range is used instead of 50 dB, so weaker signals are still shown as grey.

Cell 12 saved **bmode\_cyst\_lesion.npy** / **.png** / **\_axes.npz** and **bmode\_point\_targets.npy** / **.png** / **\_axes.npz** in output/: brightness (2078, 256) float32 in 0..1, the grayscale PNG and the axes (depth 0.0 to 40.0 mm, width -10.0 to 10.0 mm).

## 5.7 Console output

The complete console text of the run (also saved as output/console\_log.txt):

### python3 run\_team7.py

```
=====
TEAM 7 - B-MODE IMAGE FORMATION (NITK-UsoundSim)
=====
Stage order: Team 6 Image Reconstruction (Hilbert envelope + log compression)
-> Team 7 B-mode Image Formation (THIS STAGE: dynamic range, gray levels, axes, final image)
-> Team 8 Post-Image Processing (despeckling)
Folder : Team_7_B_mode_Image_Formation
Python : 3.12.4 | numpy 1.26.4 | matplotlib 3.8.3

=====
PART 1: INPUT = output of Team 6 (Image Reconstruction), copied into input/
=====
input/reconstructed_carotid_1.npz
rf          shape (1218, 128), float32, values -162577 to 166418
envelope    shape (1218, 128), float32, values 4.99583 to 170356
envelope_norm shape (1218, 128), float32, values 2.93259e-05 to 1
envelope_db shape (1218, 128), float32, values -90.655 to 0
fs          scalar float64 = 31250000.0
f0          scalar float64 = 5000000.0
c           scalar float64 = 1540.0
pitch       scalar float64 = 0.0001953
depth_start_mm scalar float64 = 5.0
input/reconstructed_carotid_2.npz
rf          shape (1218, 128), float32, values -130418 to 139393
envelope    shape (1218, 128), float32, values 7.11421 to 143632
envelope_norm shape (1218, 128), float32, values 4.95308e-05 to 1
envelope_db shape (1218, 128), float32, values -86.1025 to 0
fs          scalar float64 = 31250000.0
f0          scalar float64 = 5000000.0
c           scalar float64 = 1540.0
pitch       scalar float64 = 0.0001953
depth_start_mm scalar float64 = 5.0
[figure saved] output/input_envelope_db_overview.png (runner figure)

=====
PART 2.1: code/B_mode.ipynb, B-mode part, on reconstructed_carotid_1.npz
=====
Loaded into the namespace from the input file: rf, envelope, envelope_norm, envelope_db, fs, f0, c, pitc
↳ h, depth_start_mm
Runner sets DATA_FILE = "carotid_1_rf.npy" (only used by cell 9 to name the saved files).
Not run here (Team 6's work): cell 1 np.load, cell 3 Hilbert, cell 4 scan-line plot,
cell 5 sections 3-4 (normalise, log compression).
--- cell 1, import lines only: import numpy as np; import matplotlib.pyplot as plt
--- B_mode.ipynb code cell 2 (settings; gives dynamic_range)
--- B_mode.ipynb code cell 5, sections 5-6 (clip to dynamic range, brightness 0.1)
B-mode shape: (1218, 128)
Brightness range: 0.0 to 1.0
--- B_mode.ipynb code cell 6 (depth and width axes)
Depth: 5.0 mm to 35.0 mm
Width: -12.5 mm to 12.3 mm
--- B_mode.ipynb code cell 7 (display all steps)
[figure saved] output/carotid_1_01_all_steps.png
--- B_mode.ipynb code cell 8 (final B-mode image)
[figure saved] output/carotid_1_02_final_bmode.png
--- B_mode.ipynb code cell 9 (save, run inside output/)
Saved: bmode_carotid_1.npy | bmode_carotid_1.png | bmode_carotid_1_axes.npz
Image size: (1218, 128) (depth samples x scan lines)

=====
PART 2.2: code/B_mode.ipynb, B-mode part, on reconstructed_carotid_2.npz
=====
Loaded into the namespace from the input file: rf, envelope, envelope_norm, envelope_db, fs, f0, c, pitc
↳ h, depth_start_mm
Runner sets DATA_FILE = "carotid_2_rf.npy" (only used by cell 9 to name the saved files).
Not run here (Team 6's work): cell 1 np.load, cell 3 Hilbert, cell 4 scan-line plot,
cell 5 sections 3-4 (normalise, log compression).
--- cell 1, import lines only: import numpy as np; import matplotlib.pyplot as plt
--- B_mode.ipynb code cell 2 (settings; gives dynamic_range)
```

```

--- B_mode.ipynb code cell 5, sections 5-6 (clip to dynamic range, brightness 0..1)
B-mode shape: (1218, 128)
Brightness range: 0.0 to 1.0
--- B_mode.ipynb code cell 6 (depth and width axes)
Depth: 5.0 mm to 35.0 mm
Width: -12.5 mm to 12.3 mm
--- B_mode.ipynb code cell 7 (display all steps)
[figure saved] output/carotid_2_01_all_steps.png
--- B_mode.ipynb code cell 8 (final B-mode image)
[figure saved] output/carotid_2_02_final_bmode.png
--- B_mode.ipynb code cell 9 (save, run inside output/)
Saved: bmode_carotid_2.npy | bmode_carotid_2.png | bmode_carotid_2_axes.npz
Image size: (1218, 128) (depth samples x scan lines)

=====
PART 2.3: code/B_mode.ipynb code cells 10-12 on the simulated phantoms
=====
Input: input/reconstructed_cyst_lesion.npz, input/reconstructed_point_targets.npz
(made by our simulator, Teams 1-5, plus Team 6's envelope and log compression;
 see input/simulated/make_simulated_inputs.py)
--- cell 1, import lines only: import numpy as np; import matplotlib.pyplot as plt
--- B_mode.ipynb code cell 2 (settings; gives dynamic_range)
--- B_mode.ipynb code cell 10 (simulated: dynamic range, brightness)
cyst_lesion
  B-mode shape: (2078, 256) (depth samples x scan lines)
  Brightness range: 0.0 to 1.0
  Depth: 0.0 mm to 40.0 mm
  Width: -10.0 mm to 10.0 mm
point_targets
  B-mode shape: (2078, 256) (depth samples x scan lines)
  Brightness range: 0.0 to 1.0
  Depth: 0.0 mm to 40.0 mm
  Width: -10.0 mm to 10.0 mm
--- B_mode.ipynb code cell 11 (display the simulated images)
[figure saved] output/simulated_01_cyst_lesion_and_point_targets.png
--- B_mode.ipynb code cell 12 (save, run inside output/)
Saved: bmode_cyst_lesion.npy | bmode_cyst_lesion.png | bmode_cyst_lesion_axes.npz
Saved: bmode_point_targets.npy | bmode_point_targets.png | bmode_point_targets_axes.npz

=====
PART 3: effect of the dynamic range (runner figure, notebook formula)
=====
For DR = 40, 50, 60 dB:  $B = (\text{clip}(\text{envelope\_db}, -\text{DR}, 0) + \text{DR}) / \text{DR}$  from the input envelope_db
carotid_1, DR 40 dB: mean brightness 0.210, black (clipped) pixels 26.44 %, pixels at full white 1
carotid_1, DR 50 dB: mean brightness 0.341, black (clipped) pixels 4.90 %, pixels at full white 1
carotid_1, DR 60 dB: mean brightness 0.447, black (clipped) pixels 0.50 %, pixels at full white 1
carotid_2, DR 40 dB: mean brightness 0.168, black (clipped) pixels 30.11 %, pixels at full white 1
carotid_2, DR 50 dB: mean brightness 0.303, black (clipped) pixels 5.74 %, pixels at full white 1
carotid_2, DR 60 dB: mean brightness 0.415, black (clipped) pixels 0.63 %, pixels at full white 1
[figure saved] output/dynamic_range_40_50_60dB.png (runner figure)
60 dB image here identical to the notebook's bmode for carotid_1: True

=====
PART 4: reproducibility check (this run vs the image the notebook saved earlier)
=====
this run output/bmode_carotid_1.npy : shape (1218, 128), float32, values 0 to 1
reference input/reference/bmode_carotid_1.npy : shape (1218, 128), float32, values 0 to 1
max |this run - reference| = 6.49e-06
mean |this run - reference| = 1.11e-07
8-bit gray levels round(255 * B): 6 of 155904 pixels differ, by at most 1 level (values that sit right
  ↳ on a rounding boundary)
depth_mm and width_mm identical to reference : True
RESULT: PASS - same image (difference is float32 rounding, far below one gray level 1/255 = 0.0039)
[figure saved] output/reproducibility_check_60dB.png (runner figure)

=====
PART 5: SUMMARY
=====
carotid_1:
  input envelope_db (1218, 128), float32, values -90.655 to 0 dB
  dynamic range 60 dB -> clipped dB values -60 to 0 dB
  B-mode brightness float32, values 0 to 1, mean 0.447
  8-bit gray levels 0 to 255 (round(255 * B)): mean 114.1
  axes depth 5.0 to 35.0 mm, lateral -12.5 to 12.3 mm

```

```

    black pixels      40 dB: 26.44 %, 50 dB: 4.90 %, 60 dB: 0.50 %
carotid_2:
  input envelope_db  (1218, 128), float32, values -86.1025 to 0 dB
  dynamic range      60 dB -> clipped dB values -60 to 0 dB
  B-mode brightness  float32, values 0 to 1, mean 0.415
  8-bit gray levels  0 to 255 (round(255 * B)): mean 105.9
  axes               depth 5.0 to 35.0 mm, lateral -12.5 to 12.3 mm
  black pixels      40 dB: 30.11 %, 50 dB: 5.74 %, 60 dB: 0.63 %
cyst_lesion: B-mode (2078, 256), brightness 0 to 1, mean 0.599, depth 0.0 to 40.0 mm, lateral -10.0 to 1
↳ 0.0 mm
point_targets: B-mode (2078, 256), brightness 0 to 1, mean 0.022, depth 0.0 to 40.0 mm, lateral -10.0 to
↳ 10.0 mm
Reproducibility (carotid_1): max |this run - reference| = 6.49e-06 (PASS)

Timings:
Part 1 input overview                0.39 s
Part 2.1 notebook stage, carotid_1  0.74 s
Part 2.2 notebook stage, carotid_2  0.70 s
Part 2.3 notebook, simulated phantoms 0.57 s
Part 3 dynamic range comparison      0.45 s
Part 4 reproducibility check         0.00 s
TOTAL                               3.13 s

Files written to output/:
bmode_carotid_1.npy
bmode_carotid_1.png
bmode_carotid_1_axes.npz
bmode_carotid_2.npy
bmode_carotid_2.png
bmode_carotid_2_axes.npz
bmode_cyst_lesion.npy
bmode_cyst_lesion.png
bmode_cyst_lesion_axes.npz
bmode_point_targets.npy
bmode_point_targets.png
bmode_point_targets_axes.npz
carotid_1_01_all_steps.png
carotid_1_02_final_bmode.png
carotid_2_01_all_steps.png
carotid_2_02_final_bmode.png
dynamic_range_40_50_60dB.png
input_envelope_db_overview.png
reproducibility_check_60dB.png
simulated_01_cyst_lesion_and_point_targets.png
console_log.txt
console_screenshot_1.png, _2.png, ... (pictures of this console text, 75 lines each)

Done.
```

```

Terminal - python3 run_team7.py (part 1 of 3)

=====
TEAM 7 - B-MODE IMAGE FORMATION (NITK-UsoundSim)
=====
Stage order: Team 6 Image Reconstruction (Hilbert envelope + log compression)
-> Team 7 B-mode Image Formation (THIS STAGE: dynamic range, gray levels, axes, final image)
-> Team 8 Post-Image Processing (despeckling)
Folder : Team 7 B-mode Image Formation
Python : 3.12.4 | numpy 1.26.4 | matplotlib 3.8.3

=====
PART 1: INPUT = output of Team 6 (Image Reconstruction), copied into input/
=====
input/reconstructed_carotid_1.npz
rf          shape (1218, 128), float32, values -162577 to 166418
envelope    shape (1218, 128), float32, values 4.99583 to 170356
envelope_norm shape (1218, 128), float32, values 2.93259e-05 to 1
envelope_db  shape (1218, 128), float32, values -90.655 to 0
fs          scalar float64 = 31250000.0
f0          scalar float64 = 5000000.0
c           scalar float64 = 1540.0
pitch       scalar float64 = 0.0001953
depth_start_mm scalar float64 = 5.0
input/reconstructed_carotid_2.npz
rf          shape (1218, 128), float32, values -130418 to 139393
envelope    shape (1218, 128), float32, values 7.11421 to 143632
envelope_norm shape (1218, 128), float32, values 4.95308e-05 to 1
envelope_db  shape (1218, 128), float32, values -86.1025 to 0
fs          scalar float64 = 31250000.0
f0          scalar float64 = 5000000.0
c           scalar float64 = 1540.0
pitch       scalar float64 = 0.0001953
depth_start_mm scalar float64 = 5.0
[figure saved] output/input_envelope_db_overview.png (runner figure)

=====
PART 2.1: code/B_mode.ipynb, B-mode part, on reconstructed_carotid_1.npz
=====
Loaded into the namespace from the input file: rf, envelope, envelope_norm, envelope_db, fs, f0, c, pitch, depth_start_mm
Runner sets DATA FILE = "carotid_1_rf.npy" (only used by cell 9 to name the saved files).
Not run here (Team 6's work): cell 1 np.load, cell 3 Hilbert, cell 4 scan-line plot,
cell 5 sections 3-4 (normalise, log compression).
--- cell 1, import lines only: import numpy as np; import matplotlib.pyplot as plt
--- B_mode.ipynb code cell 2 (settings; gives dynamic range)
--- B_mode.ipynb code cell 5, sections 5-6 (clip to dynamic range, brightness 0..1)
B-mode shape: (1218, 128)
Brightness range: 0.0 to 1.0
--- B_mode.ipynb code cell 6 (depth and width axes)
Depth: 5.0 mm to 35.0 mm
Width: 12.5 mm to 12.3 mm
--- B_mode.ipynb code cell 7 (display all steps)
[figure saved] output/carotid_1_01_all_steps.png
--- B_mode.ipynb code cell 8 (final B-mode image)
[figure saved] output/carotid_1_02_final_bmode.png
--- B_mode.ipynb code cell 9 (save, run inside output/)
Saved: bmode_carotid_1.npy | bmode_carotid_1.png | bmode_carotid_1_axes.npz
Image size: (1218, 128) (depth samples x scan lines)

=====
PART 2.2: code/B_mode.ipynb, B-mode part, on reconstructed_carotid_2.npz
=====
Loaded into the namespace from the input file: rf, envelope, envelope_norm, envelope_db, fs, f0, c, pitch, depth_start_mm
Runner sets DATA FILE = "carotid_2_rf.npy" (only used by cell 9 to name the saved files).
Not run here (Team 6's work): cell 1 np.load, cell 3 Hilbert, cell 4 scan-line plot,
cell 5 sections 3-4 (normalise, log compression).
--- cell 1, import lines only: import numpy as np; import matplotlib.pyplot as plt
--- B_mode.ipynb code cell 2 (settings; gives dynamic range)
--- B_mode.ipynb code cell 5, sections 5-6 (clip to dynamic range, brightness 0..1)
B-mode shape: (1218, 128)
Brightness range: 0.0 to 1.0
--- B_mode.ipynb code cell 6 (depth and width axes)
Depth: 5.0 mm to 35.0 mm
Width: 12.5 mm to 12.3 mm
--- B_mode.ipynb code cell 7 (display all steps)
[figure saved] output/carotid_2_01_all_steps.png

```

output/console\_screenshot\_1.png: picture of part 1 of 3 of the console output (75 lines per picture).

```

Terminal - python3 run_team7.py (part 2 of 3)

--- B mode.ipynb code cell 8 (final B-mode image)
[Figure saved] output/carotid_2_02_final_bmode.png
--- B mode.ipynb code cell 9 (save, run inside output/)
Saved: bmode_carotid_2.npy | bmode_carotid_2.png | bmode_carotid_2_axes.npz
Image size: (1218, 128) (depth samples x scan lines)

=====
PART 2.3: code/B mode.ipynb code cells 10-12 on the simulated phantoms
=====
Input: input/reconstructed_cyst_lesion.npz, input/reconstructed_point_targets.npz
(made by our simulator, Teams 1-5, plus Team 6's envelope and log compression;
see input/simulated/make_simulated_inputs.py)
--- cell 1, import lines only: import numpy as np; import matplotlib.pyplot as plt
--- B mode.ipynb code cell 2 (settings; gives dynamic_range)
--- B mode.ipynb code cell 10 (simulated: dynamic range, brightness)
cyst_lesion
B-mode shape: (2078, 256) (depth samples x scan lines)
Brightness range: 0.0 to 1.0
Depth: 0.0 mm to 40.0 mm
Width: -10.0 mm to 10.0 mm
point_targets
B-mode shape: (2078, 256) (depth samples x scan lines)
Brightness range: 0.0 to 1.0
Depth: 0.0 mm to 40.0 mm
Width: -10.0 mm to 10.0 mm
--- B mode.ipynb code cell 11 (display the simulated images)
[Figure saved] output/simulated_01_cyst_lesion_and_point_targets.png
--- B mode.ipynb code cell 12 (save, run inside output/)
Saved: bmode_cyst_lesion.npy | bmode_cyst_lesion.png | bmode_cyst_lesion_axes.npz
Saved: bmode_point_targets.npy | bmode_point_targets.png | bmode_point_targets_axes.npz

=====
PART 3: effect of the dynamic range (runner figure, notebook formula)
=====
For DR = 40, 50, 60 dB: B = (clip(envelope_db, -DR, 0) + DR) / DR from the input envelope_db
carotid_1, DR 40 dB: mean brightness 0.210, black (clipped) pixels 26.44 %, pixels at full white 1
carotid_1, DR 50 dB: mean brightness 0.341, black (clipped) pixels 4.90 %, pixels at full white 1
carotid_1, DR 60 dB: mean brightness 0.447, black (clipped) pixels 0.50 %, pixels at full white 1
carotid_2, DR 40 dB: mean brightness 0.168, black (clipped) pixels 30.11 %, pixels at full white 1
carotid_2, DR 50 dB: mean brightness 0.303, black (clipped) pixels 5.74 %, pixels at full white 1
carotid_2, DR 60 dB: mean brightness 0.415, black (clipped) pixels 0.63 %, pixels at full white 1
[figure saved] output/dynamic_range_40_50_60dB.png (runner figure)
60 dB image here identical to the notebook's bmode for carotid_1: True

=====
PART 4: reproducibility check (this run vs the image the notebook saved earlier)
=====
this run output/bmode_carotid_1.npy : shape (1218, 128), float32, values 0 to 1
reference input/reference/bmode_carotid_1.npy : shape (1218, 128), float32, values 0 to 1
max [this run - reference] = 6.49e-06
mean [this run - reference] = 1.11e-07
8-bit gray levels round(255 * B): 6 of 155904 pixels differ, by at most 1 level (values that sit right on a rounding bounda
ry)
depth mm and width mm identical to reference : True
RESULT: PASS - same image (difference is float32 rounding, far below one gray level 1/255 = 0.0039)
[figure saved] output/reproducibility_check_60dB.png (runner figure)

=====
PART 5: SUMMARY
=====
carotid_1:
input envelope_db (1218, 128), float32, values -90.655 to 0 dB
dynamic range 60 dB -> clipped dB values -60 to 0 dB
B-mode brightness float32, values 0 to 1, mean 0.447
8-bit gray levels 0 to 255 (round(255 * B)): mean 114.1
axes depth 5.0 to 35.0 mm, lateral -12.5 to 12.3 mm
black pixels 40 dB: 26.44 %, 50 dB: 4.90 %, 60 dB: 0.50 %
carotid_2:
input envelope_db (1218, 128), float32, values -86.1025 to 0 dB
dynamic range 60 dB -> clipped dB values -60 to 0 dB
B-mode brightness float32, values 0 to 1, mean 0.415
8-bit gray levels 0 to 255 (round(255 * B)): mean 105.9
axes depth 5.0 to 35.0 mm, lateral -12.5 to 12.3 mm
black pixels 40 dB: 30.11 %, 50 dB: 5.74 %, 60 dB: 0.63 %
cyst_lesion: B-mode (2078, 256), brightness 0 to 1, mean 0.599, depth 0.0 to 40.0 mm, lateral -10.0 to 10.0 mm

```

output/console\_screenshot\_2.png: picture of part 2 of 3 of the console output (75 lines per picture).

```

Terminal - python3 run_team7.py (part 3 of 3)

point targets: B-mode (2078, 256), brightness 0 to 1, mean 0.022, depth 0.0 to 40.0 mm, lateral -10.0 to 10.0 mm
Reproducibility (carotid_1): max |this run - reference| = 6.49e-06 (PASS)

Timings:
Part 1 input overview                0.39 s
Part 2.1 notebook stage, carotid_1  0.74 s
Part 2.2 notebook stage, carotid_2   0.70 s
Part 2.3 notebook, simulated phantoms 0.57 s
Part 3 dynamic range comparison       0.45 s
Part 4 reproducibility check          0.00 s
TOTAL                                3.13 s

Files written to output/:
bmode_carotid_1.npy
bmode_carotid_1.png
bmode_carotid_1_axes.npz
bmode_carotid_2.npy
bmode_carotid_2.png
bmode_carotid_2_axes.npz
bmode_cyst_lesion.npy
bmode_cyst_lesion.png
bmode_cyst_lesion_axes.npz
bmode_point_targets.npy
bmode_point_targets.png
bmode_point_targets_axes.npz
carotid_1_01_all_steps.png
carotid_1_02_final_bmode.png
carotid_2_01_all_steps.png
carotid_2_02_final_bmode.png
dynamic_range_40_50_60dB.png
input_envelope_db_overview.png
reproducibility_check_60dB.png
simulated_01_cyst_lesion_and_point_targets.png
console_log.txt
console_screenshot_1.png, _2.png, ... (pictures of this console text, 75 lines each)

Done.

```

output/console\_screenshot\_3.png: picture of part 3 of 3 of the console output (75 lines per picture).

## 6 How to run

Step by step, for macOS, Linux and Windows

1. Install Python 3 (3.9 or newer) from <https://www.python.org>. On Windows tick 'Add Python to PATH'.
2. Open a terminal: macOS Terminal, Linux terminal, or Windows Command Prompt / PowerShell.
3. Go into the folder: **cd path/to/Team\_7\_B\_mode\_Image\_Formation** (Windows: `cd path\to\Team_7_B_mode_Image_Formation`).
4. Install the packages once: **python3 -m pip install -r requirements.txt** (Windows: `python -m pip install -r requirements.txt`).
5. Run: **python3 run\_team7.py** (Windows: `python run_team7.py`).
6. Look at the results in **output/** (PNG figures, `bmode_*.npy/.png/_axes.npz`, `console_log.txt`, console pictures).

**Expected runtime:** about 3 seconds (the script measured 3.13 s on a Mac laptop; a first run can take a few seconds longer while libraries load). No windows open: all figures are saved as files.

**Open the notebook yourself (Jupyter):** `B_mode.ipynb` can also be opened on its own (python3 -m pip install jupyter scipy, then jupyter notebook; or VS Code with the Jupyter extension, or Google Colab). Run all cells. It runs the whole chain including Team 6's part (Hilbert envelope, log compression) and then this team's steps, starting from the raw RF. For this, **code/carotid\_1\_rf.npy** (a copy of the raw carotid RF) is placed next to the notebook; the notebook's `DATA_FILE` already points to it, and it saves `bmode_carotid_1.*` next to the notebook. In Colab, upload `carotid_1_rf.npy` next to the notebook. `carotid_2_rf.npy` is not in this folder. `run_team7.py` does not use this RF file; it starts from Team 6's `.npz` handoff in `input/`. The added code cells 10-12 also show and save the simulated cyst lesion and point targets; they read `../input/reconstructed_cyst_lesion.npz` and `../input/reconstructed_point_targets.npz`, so keep `input/` next to `code/`. In Colab, upload these two files as well and change the two paths in `SIM_FILES` (code cell 10) to just the file names.

## Troubleshooting

| Problem                                              | What to do                                                                             |
|------------------------------------------------------|----------------------------------------------------------------------------------------|
| No module named numpy / matplotlib                   | Run the pip install step again with the same python3 / python command                  |
| python3 not found (Windows)                          | Use python instead of python3                                                          |
| FileNotFoundError:<br>...reconstructed_carotid_1.npz | Run from inside the folder; keep <code>input/</code> next to <code>run_team7.py</code> |
| No picture windows                                   | Normal: figures are saved in <code>output/</code>                                      |
| Numbers differ by about 1e-6                         | Different numpy versions round float32 slightly differently                            |

## 7 Check yourself

Questions with short answers

### What exactly does Team 7 receive from Team 6, and what is 0 dB?

The log-compressed envelope  $\text{envelope\_db} = 20 \cdot \log_{10}(E/\max E + 1e-12)$  for every pixel, not yet clipped. 0 dB is the strongest echo of the frame; all other values are negative (down to -90.655 dB for carotid\_1).

### What does the dynamic range do?

It decides how many dB below the peak are displayed. Values below -DR are clipped to -DR and become black; the range -DR..0 dB is spread over black..white.

### What happens to the image when DR goes from 60 dB to 40 dB?

Higher contrast and a darker image: more weak echoes are clipped. In this run the black pixels of carotid\_1 rise from 0.50 % to 26.44 % and the mean brightness falls from 0.447 to 0.210.

### With DR = 60 dB, what brightness and grey level does a pixel at -45 dB get?

$B = (-45 + 60)/60 = 0.25$ , grey level  $\text{round}(255 \cdot 0.25) = 64$  (dark grey).

### How is depth computed, and why the factor 2?

$z = z_0 + c \cdot i/(2 \cdot fs)$ .  $i/fs$  is the time since the start; the sound travels down and back, so the reflector is at half the distance. Here 5.0 to 35.0 mm.

### Where is the lesion in the simulated cyst image, and why is it dark?

At about 24-36 mm depth and  $x = -6$  to  $+6$  mm (a 6 mm radius cyst centred at 30 mm). It is anechoic: the phantom has no scattering inside it, so little echo comes back and the 60 dB mapping shows it dark.

### Why does the saved PNG look tall and thin, but the figure does not?

The array has 1218 rows and 128 columns (one pixel per sample). The figure uses extent in mm, so 30 mm of depth and 24.8 mm of width are drawn at their true size.